Multimedia Communications

Lecture Notes

A.Y. 2025/2026

Contents

1 Multimedia Representation & Perception **7**

1.1 Human Eye . . . 7
1.1.1 CSF - Contrast Sensitivity Function . . . 7
1.1.2 Eye Sensitivity . . . 7
1.2 Color Spaces . . . 7
1.2.1 RGB . . . 7
1.2.2 HSV . . . 7
1.2.3 YCbCr = YUV . . . 7
1.3 Machine-centric Multimedia . . . 7
1.4 Frequency Masking Functions . . . 7
1.5 Image and Video Representation . . . 8
1.5.1 Images . . . 8
1.5.2 Color Sampling . . . 8
1.6 Compression . . . 8
1.6.1 Lossless Techniques . . . 8
1.6.2 Lossy Techniques . . . 9
1.7 Quantization . . . 9
1.7.1 Quality Evaluation . . . 9
1.7.2 Peak SNR (PSNR) . . . 9
1.7.3 Weighted Peak SNR (WPSNR) . . . 9
1.7.4 Other criteria . . . 9
1.7.5 Criteria to evaluate a compression algorithm . . . 10

2 Scalar and Predictive Quantization **11**

2.1 Scalar Quantizer . . . 11
2.1.1 Rate . . . 11
2.1.2 Distortion . . . 11
2.2 Uniform Quantization . . . 11
2.2.1 Definitions . . . 11
2.2.2 Unsigned Data . . . 12
2.2.3 Signed Data (Midtread) . . . 12
2.2.4 Dead-zone Quantization for Signed Data . . . 12
2.2.5 Rate and Distorsion curve . . . 12
2.3 Optimal Quantization . . . 13
2.3.1 High Resolution (HR) Uniform Quantization . . . 13
2.3.2 Lloyd-Max algorithm . . . 13
2.3.3 Real-life Lloyd-Max adaptation . . . 14
2.4 Predictive Scalar Quantization . . . 14
2.4.1 Sparsification . . . 14

2.4.2 Prediction . . . 14
2.4.3 Prediction Error . . . 14
2.4.4 Coding Gains . . . 15
2.4.5 Linear Predictors . . . 15
2.4.6 Optimal Predictor (Wiener-Hopf) . . . 15
2.4.7 Worked example: AR(1) Gaussian signal . . . 15
2.4.8 Prediction Gain: filter order impact . . . 16
2.4.9 Local adaptation . . . 16
2.4.10 Wrong scheme: open-loop prediction (drift) . . . 16
2.4.11 Correct scheme: closed-loop prediction (DPCM) . . . 17
2.4.12 Impact of entropy coding on PQ . . . 18
2.4.13 Direct vs Predictive Quantization . . . 18

3 Lossless Coding **18**
3.1 Definitions . . . 18
3.2 Code Types . . . 18
3.2.1 Fixed-Length Code . . . 18
3.2.2 Variable-Length Code . . . 19
3.3 Variable-length codes theorems . . . 19
3.3.1 McMillan's Theorem . . . 19
3.3.2 Kraft's Inequality . . . 19
3.3.3 Proof for Kraft's $\Rightarrow$ (necessity) . . . 19
3.3.4 Proof for Kraft's $\Leftarrow$ (sufficiency) . . . 19
3.4 Recall: Information Theory . . . 20
3.4.1 Information . . . 20
3.4.2 Source Entropy . . . 20
3.4.3 Max Entropy Distribution . . . 20
3.4.4 Joint Entropy . . . 20
3.4.5 Conditional Entropy . . . 20
3.5 Optimal Code . . . 21
3.5.1 Math Formulation . . . 21
3.5.2 Shannon's source coding theorem . . . 21
3.5.3 Huffman Coding . . . 21
3.5.4 Block Coding . . . 21
3.5.5 Limits of Huffman . . . 22
3.5.6 Arithmetic coding . . . 22
3.5.7 Adaptivity and Context-based coding . . . 22
3.5.8 Why Arithmetic is preferred over Huffman (wrap-up) . . . 23
3.5.9 Context-based coding . . . 23
3.6 Other Techniques . . . 23
3.6.1 Unsigned Exp-Golomb . . . 23
3.6.2 Signed Exp-Golomb . . . 23
3.6.3 Dictionary-based coding . . . 23
3.6.4 LZW coding . . . 23
3.6.5 Examples of real-life Coding . . . 24
3.6.6 Neural Lossless Coding (NLC) Techniques . . . 24
3.6.7 Guidelines to reach $\mathcal{L} \approx \mathcal{H}$ . . . 24

4 Transform Coding and JPEG **24**
4.1 Introduction with Block Coding . . . 24
4.2 Huang-Schulteiss (HS) formula . . . 25
4.2.1 HS formula derivation . . . 25
4.2.2 HS formula interpretation . . . 25

4.2.3 Why the Geometric Mean is the key quantity . . . 25
4.3 Transform Coding . . . 26
4.3.1 Orthogonal Transforms . . . 26
4.3.2 Orthogonal Transforms applied to Block Coding . . . 26
4.3.3 Transform Coding example . . . 26
4.4 Practical Algorithms for Resource Allocation . . . 26
4.4.1 Greedy Algorithm . . . 26
4.4.2 Modified HS algorithm . . . 27
4.5 Towards Optimal Transform . . . 27
4.5.1 Karhunen-Loève Transform (KLT) . . . 27
4.6 Frequency Transforms . . . 27
4.6.1 1D-DFT for Compression . . . 27
4.6.2 2D-DFT for Compression . . . 28
4.6.3 DFT Separable Implementation . . . 28
4.6.4 DCT Transform Matrix . . . 28
4.6.5 1D-DCT Transform . . . 28
4.6.6 2D-DCT Transform: Block-Based DCT . . . 28
4.7 JPEG . . . 28
4.7.1 Encoding Strategy . . . 29
4.7.2 Quantization-step table . . . 29
4.7.3 Scaling Factor and Quality . . . 29
4.7.4 Zig-Zag Coding . . . 29
4.8 Entropy Coding of the Coefficients . . . 30
4.8.1 DC Coefficients . . . 30
4.8.2 AC Coefficients . . . 30
4.9 Frame Building . . . 30
4.10 JPEG additional informations . . . 30
4.10.1 JFIF: JPEG File Interchange Format . . . 30
4.10.2 EXIF: Extended Image File Format . . . 30

5 Wavelet-based Image Compression 31
5.1 Signal Analysis . . . 31
5.1.1 Signal Analysis through Projection . . . 31
5.1.2 Resolution Tradeoff: Time-Frequency Heisenberg-like Uncertainty principle . . . 31
5.1.3 Frequency Analysis: STFT and Rigid Tilting . . . 31
5.2 Discrete Wavelet Transform (DWT) and Multi Resolution Analysis (MRA) . . . 31
5.2.1 1D-MRA . . . 31
5.2.2 2D-MRA . . . 31
5.2.3 EZW: Embedded Zerotrees of Wavelet Coefficients . . . 32
5.2.4 Recall: Bitplanes . . . 32
5.2.5 EZW Algorithm . . . 32
5.3 JPEG2000 . . . 32
5.3.1 Introduction . . . 32
5.3.2 Quantization . . . 33
5.3.3 Comparison between JPEG and JP2K (JPEG2000) . . . 33
5.3.4 Error Robustness in compressed data . . . 33
5.4 Conceptual Maps . . . 33

6 Learned Image Coding (LIC) / Neural Image Coding (NIC) 34
6.1 Coding Architecture . . . 34
6.1.1 Optimization of the Loss Function . . . 34
6.2 NN Recap . . . 34
6.2.1 Gradient Descent and Backpropagation . . . 34

6.2.2 GDN: Generalized Divisive Normalization . . . 34
6.3 JPEG-AI standard . . . 34
6.3.1 Core Idea . . . 34
6.3.2 Auto-Encoder Framework . . . 34
6.3.3 Rate-Distorsion Variable Auto-Encoder . . . 35
6.3.4 Scale Hyperprior: spatial adaptability . . . 35
6.3.5 Hierarchical VAE . . . 35
6.3.6 Conclusions . . . 35

7 Motion Estimation **36**
7.1 Variational Methods . . . 36
7.1.1 Velocity Vector Field - Optical Flow . . . 36
7.1.2 Motion Vector Field . . . 36
7.1.3 Data Attachment and Regularization . . . 36
7.1.4 Optical Flow Problem . . . 36
7.1.5 Optical Flow - Displacement Field . . . 36
7.1.6 Optical Flow - Constant Illumination Hypothesis (CIH) . . . 36
7.1.7 Optical Flow - Equation . . . 37
7.1.8 Optical Flow - Solution: Horn & Schunck method . . . 37
7.2 Block Matching . . . 37
7.2.1 Formulation . . . 37
7.2.2 Evaluation of the MVF . . . 38
7.2.3 Block Matching criteria . . . 38
7.2.4 Full-Search research strategy . . . 39
7.2.5 Fast Research Methods: 3SS (Three step search, 2D-log) . . . 39
7.2.6 Fast Research Methods: Diamond Search . . . 39
7.2.7 Fast Research Methods: Hex Search . . . 39
7.2.8 Fast Research Methods: TZSearch . . . 39
7.2.9 BM Improvement: Sub-pixel precision . . . 39
7.2.10 BM Improvement: Variable Blocksize . . . 39
7.3 Parametric Methods . . . 40
7.3.1 Affine Model . . . 40
7.3.2 Best possible parameters . . . 40
7.4 Deep-Learning for Motion Estimation . . . 40
7.4.1 Time evolution . . . 40
7.4.2 Usage Comparison . . . 41

8 Video-coding Principles **42**
8.1 Block Schema . . . 42
8.2 Block-Matching motion estimation . . . 42
8.2.1 Motion Compensation . . . 42
8.2.2 Adaptive block coding . . . 42
8.2.3 Design Parameters . . . 43
8.3 GOP: Group of Pictures . . . 43
8.3.1 'I' frames: Intra Frames . . . 43
8.3.2 'B' frames: 'between' frames . . . 43
8.3.3 'P' frames: Predictive Frames . . . 43
8.3.4 Rate and quality of the frame types . . . 43
8.4 Hybrid Video Encoder . . . 44
8.4.1 Frame coding modes . . . 44
8.4.2 Block Size: Block Partition Problem . . . 44
8.4.3 Coding Mode i_k selection . . . 44
8.4.4 Encoder scheme . . . 45

8.5 Hybrid Video Decoder . . . 46
8.5.1 Decoder scheme . . . 46
8.5.2 Key elements . . . 46
8.6 Video Encoding Standards . . . 47
8.6.1 MPEG-1 . . . 47

9 Modern Video-Compression Standards 48
9.1 Universal Hybrid Video Encoder . . . 48
9.1.1 Codecs applications . . . 48
9.2 Rate-Distorsion Optimization . . . 49
9.2.1 Block Partitioning Problem: H.264 . . . 49
9.2.2 Block Partitioning Problem: H.265/HEVC . . . 49
9.2.3 Block Partitioning Problem: H.266/VVC . . . 49
9.3 Spatial and Temporal Frame Prediction . . . 49
9.3.1 Intra Prediction . . . 49
9.3.2 Inter Prediction . . . 50
9.3.3 Merge Mode . . . 50
9.4 Filtering, Transforms and Quantization . . . 50
9.4.1 Residual Coding . . . 50
9.4.2 In-loop Deblocking Filter . . . 50
9.4.3 Slices . . . 51
9.5 Network parallelism . . . 51
9.5.1 NALU . . . 51

10 Audio and Speech Coding 54
10.1 Audio Modeling . . . 54
10.1.1 Requirements . . . 54
10.1.2 Vowels . . . 54
10.1.3 Music . . . 54
10.1.4 Human Speech Production System . . . 54
10.2 Linear Predictive Coding . . . 54
10.2.1 Yule-Walker Equation . . . 55
10.2.2 Residuals . . . 55
10.2.3 LPC Synthesis . . . 55
10.3 Vector Quantization . . . 56
10.3.1 CELP Paradigm (Code-Excited Linear Prediction) . . . 56
10.3.2 Perceptual Coding for Audio . . . 57
10.4 Modern Techniques . . . 58
10.4.1 OPUS . . . 58
10.4.2 Neural Codecs . . . 58
10.4.3 Spatial Audio . . . 58
10.4.4 Quality Assessment . . . 58

11 Quality Assessment and QoE for Multimedia Services 59
11.1 What is Quality Assessment? . . . 59
11.2 Image Quality: Subjective Assessment . . . 59
11.2.1 Testing Conditions . . . 59
11.2.2 1^{st} Technique: Single Stimulus . . . 59
11.2.3 2^{nd} Technique: Double Stimulus . . . 59
11.2.4 3^{rd} Technique: Pairwise Comparison . . . 59
11.3 Human Assessment Measures . . . 59
11.3.1 MOS: Mean Opinion Score . . . 59
11.3.2 Standard Measures . . . 60

11.4 Image Quality: Objective Assessment . . . 60
11.4.1 Full Reference Techniques . . . 60
11.4.2 Y component measures . . . 60
11.4.3 CbCr component measures . . . 60
11.4.4 Total PSNR measure . . . 60
11.4.5 Bjontegaard Deltas . . . 60
11.4.6 Perceptual Metrics . . . 61
11.4.7 Reduced Reference . . . 61
11.4.8 No Reference . . . 61

12 Adaptive Streaming **62**
12.1 Video Content Distribution . . . 62
12.1.1 Challenges . . . 62
12.1.2 Coding Rate Bottleneck . . . 62
12.1.3 Paradigms . . . 62
12.2 Scalable Video Coding (SVC) . . . 63
12.2.1 Scalability Types . . . 63
12.2.2 Smart Layer Pruning . . . 63
12.2.3 Streaming via Legacy HTTP . . . 64
12.2.4 QUIC and HTTP/3 . . . 64
12.3 Adaptive Bitrate Streaming . . . 64
12.3.1 Media Presentation Description (MPD) file . . . 64
12.3.2 Network Metrics: Throughput . . . 65
12.3.3 Network Metrics: E-2-E Nodal Delay . . . 65
12.4 Buffer Dynamics . . . 65
12.4.1 Rebuffering event . . . 65
12.4.2 Playout management . . . 66
12.4.3 Client Scoring-Policy Design . . . 67
12.5 ABR Strategies . . . 67
12.5.1 Throughput-based ABR . . . 67
12.5.2 Buffer-based ABR . . . 67
12.5.3 Hybrid ABR strategy . . . 67
12.5.4 Learning-Based ABR . . . 68
12.5.5 Multiplayer Competition . . . 68
12.5.6 Neural Video Coding . . . 68

1 Multimedia Representation & Perception

1.1 Human Eye

1.1.1 CSF - Contrast Sensitivity Function

The contrast sensitivity function (CSF) describes how visible a spatial pattern is as its spatial frequency changes. It is usually plotted as sensitivity, or equivalently the minimum visible contrast, against spatial frequency measured in cycles per degree of visual angle. Draw a chart with on the x-axis the pixels per cycle in decreasing order, and on the y-axis the percentage of contrast, we'll see that at the higher corners we are not able to distinguish lines. This is position and frequency dependent.

1.1.2 Eye Sensitivity

Rods have Sensitivity to brightness, Cones can sense colors. We have 3 types of cones:

- Blue (445 nm) cones, 2% of the total
- Green (535 nm) cones, 33% of the total
- Red (575 nm) cones, 65% of the total

1.2 Color Spaces

1.2.1 RGB

RGB is an additive color space in which each color is represented by the intensity of red, green and blue light. A pixel therefore contains three numerical components, one per channel. We use 3 channels: Red, Green and Blue to identify each colour. Since we use 8 bits we have $2^8 = 256$ values. This means $256^3 = 16.7$ million possible colors.

1.2.2 HSV

HSV is a perceptual color representation that separates color type (hue), color purity (saturation), and brightness (value). Unlike RGB, its coordinates are closer to the terms used by people to describe colors. Hue, Saturation and Value.
By applying a rotation on the 3D cube identifiying the RGB space, we put the black on $z = -1$ and white on $z = 1$, we get another space.
z is the luminance axis. The longitude corresponds to the color (hue), while the radius is the saturation.

1.2.3 YCbCr = YUV

YCbCr is a color representation that separates luminance-like information Y from two chrominance components Cb and Cr. This separation allows chroma to be sampled or quantized more coarsely than luminance with limited perceived quality loss. Y = Luminance, Cb and Cr (U and V) are the chrominance informations. (Polar coordinates)
Similarly to HSV, we have luminance on the z-axis, while Chrominance infos are respectively saturation and value. This is a more natural way to identify colors. (exmaple: Bright (z-axis) Magenta (angle) Pastel (radius)).
Knowing this we can lose some informations about chrominance, thus getting a **visually** lossless image (sampling 1/4 of CbCr, but keeping all pixel[1] Y sampled).

[1] Pixel = Picture Element

1.3 Machine-centric Multimedia

Given that contents are consumed by people and machines, some information that for us is irrelevant could be fatal to the machines if not available, like dropping CbCr. We must know the important features.

1.4 Frequency Masking Functions

Frequency masking is a perceptual effect in which a strong spectral component makes nearby weaker components difficult or impossible to hear. A masking function specifies the hearing threshold around each masker as a function of frequency. $S_m(f_i, \sigma_i^2, f)$ functions. We don't encode information that couldn't be heard, like in mp3/aac/... codecs.
Basically if 2 sinewaves S_m are adjacent, we hardly distinguish them, while if they are somewhat far it is easy to distinguish them. Frequency analysis is very used in this field, still the best technique (Fourier analysis).

1.5 Image and Video Representation

1.5.1 Images

Images are stored as a $N \times M$ matrix ($f_{n,m}$ with $n \in \{0 \cdots N-1\}, m \in \{0 \cdots M-1\}$).
Sometimes can be useful to use a single index, we define $k = (n-1)M + m \rightarrow$ image f_k.

1.5.2 Color Sampling

Color sampling specifies how many chrominance samples are retained relative to luminance samples. The notation $J : a : b$ describes horizontal and vertical chroma sampling over a reference region of J luminance samples. There are 4 ways to sample color, following the `J:a:b` schema
(J = reference horizontal size, a = chroma samples on 1^{st} line, b = chroma samples on 2^{nd} line)
Common schemes are (J = 4 columns):

- 4:1:1 $\rightarrow$ 1° row is 1 sample (1-4), 2° row is another sample
  $\Rightarrow$ full vertical and 1/4 horizontal resolution
- **Most Used 4:2:0** $\rightarrow$ 1° row has 2 samples (1-2 + 3-4), 2° row reuses the samples (0 further).
  $\Rightarrow$ half vertical and half horizontal resolution
- 4:2:2 $\rightarrow$ 4:2:0 but sampled also on second line
  $\Rightarrow$ full vertical and half horizontal resolution
- 4:4:4 $\rightarrow$ complete sampling, full resolution

1.6 Compression

Compression is the representation of the same source content with fewer bits by removing statistical redundancy and, in lossy systems, perceptually less important information. Compressed size is measured in bits or bytes; coding rate is commonly measured in bits per pixel (bpp) for images and bits per second (bit/s) for timed media. A frame I in a DVB system is identified as:

$$I : (n, m, T, c) \rightarrow x \in \left\{0, \cdots, 2^b - 1\right\} \text{ with } \begin{cases} n = \text{ row} \\ m = \text{ column} \\ T = \text{ frame period} \\ c = \text{ channel} \end{cases}$$

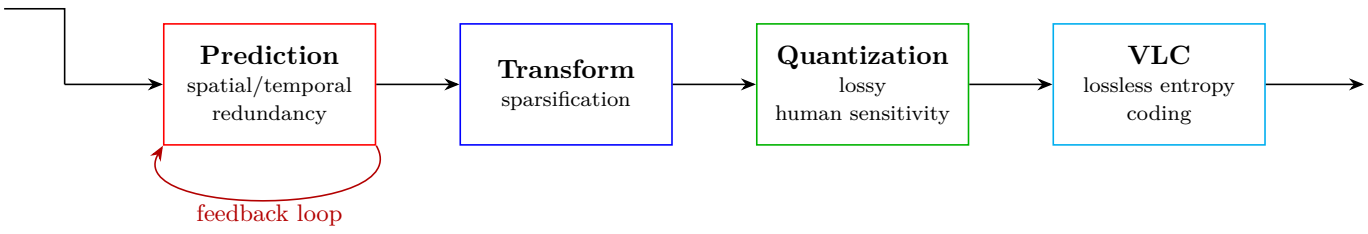
We need compression, keyword: redundancy (delete all unnecessary information), there are 3 types:

- Spatial and Temporal redundancy (neighbor-pixels are similar)
- Human Sensitivity redundancy (psychovisual)
- Semantical redundancy (images described by words)

Compression ratio is the uncompressed size divided by the compressed size:

$$\text{Compression Ratio} = \frac{B_{in}}{B_{out}}.$$

A ratio of 10 : 1, for example, means that the compressed representation uses one tenth of the original number of bits.



1.6.1 Lossless Techniques

- Bit-by-bit identical representation
- useful for X-rays, and medical images.
- not so much compression

1.6.2 Lossy Techniques

- decoded $\neq$ original
- we compress much more
- visually lossy compression + then lossless to squeeze even more

1.7 Quantization

Quantization is a lossy mapping from a large or continuous set of values to a finite set of reconstruction levels. It reduces the number of bits needed to describe samples, at the cost of quantization error. We need to have signal sparsification (information is concentrated in specific samples while other are not so much different).

1.7.1 Quality Evaluation

We need to define some parameters, given f original image matrix, and $\tilde{f}$ the reconstructed one, we can define the Error Image: $\xi(f, \tilde{f}) = f - \tilde{f}$

1.7.2 Peak SNR (PSNR)

PSNR is a logarithmic objective quality measure comparing peak signal power with mean squared reconstruction error. It is expressed in decibels (dB); a higher value means lower pixel-domain error, although not necessarily better perceived quality.

$$PSNR(f, \tilde{f}) = 10 \log_{10} \left(\frac{V^2}{D(f, \tilde{f})} \right)$$

where D is the MSE of the difference: $D(f, \tilde{f}) = \frac{1}{NM} ||\xi(f, \tilde{f})||^2$, and V is the max value which a color can be. Typical values:

- over 40: perfect image, lossless
- equal 40: very high quality
- between 30 and 40: some errors visible
- below 30: problems

1.7.3 Weighted Peak SNR (WPSNR)

WPSNR is a perceptually weighted version of PSNR. Before computing error energy, it filters the error image so that visually important distortions contribute more than less visible ones; its result is also expressed in dB. Instead of D we use Weighted D $D_W(f, \tilde{f}) = \frac{1}{NM} ||h \star \xi(f, \tilde{f})||^2$, such that:

$$WPSNR(f, \tilde{f}) = 10 \log_{10} \left(\frac{V^2}{D_W(f, \tilde{f})} \right)$$

1.7.4 Other criteria

- SSIM: Percepted quality metric

$$SSIM(x,y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)} \in [0,1]$$

  Product of 3 components: **luminance** (μ's), **contrast** (σ's), **structure** (σ_{xy}). Computed per block, then averaged. Captures what MSE misses: same MSE can give very different perceived quality (contour errors least visible, high-frequency noise most visible).

- LPIPS: Neural-Network based quality metrix (Deep-Perceptual Metric), distance in feature space of a pre-trained NN instead of pixel space. Low PSNR can coexist with high perceptual plausibility.

PSNR/SSIM are objective proxies; the ground truth is the human observer (**subjective criteria**): expensive and slow, so NP-OC are used in practice. Modern approach: report all three (PSNR, SSIM, LPIPS).

1.7.5 Criteria to evaluate a compression algorithm

Beyond **rate** (compression ratio, bpp) and **quality** (distortion metrics above), three more axes:

- **Complexity** is the computational and memory work required by encoding or decoding. It is measured through operation counts, execution time, memory use, or energy consumption, depending on the application.

- **Delay** is the time between receiving source data and producing usable coded or decoded data. It is measured in seconds or milliseconds and is driven by buffering, look-ahead, coding order, and processing time.

- **Robustness** is the ability of a compressed representation to limit quality loss when bits or packets are corrupted or missing.
  In uncompressed images a bit flip stays local; with compression there is **error propagation** (problem for noisy channels)

Design tensions: quality ↑ conflicts with rate ↓; robustness ↑ conflicts with complexity ↓ and delay ↓. Modern design balances all five.

2 Scalar and Predictive Quantization

Quantization is the replacement of each input value with one value selected from a finite reconstruction set. It controls the number of symbols, and therefore the number of bits, used to represent samples while introducing a measurable approximation error.
θ^i are the quantization region/cells $= (t^i, t^{i+1})$
There is a total of $(L+1)$ thresholds and L levels, total of $2L+1$ elements. Mathematical definition:

$$Q : x \in \mathbb{R} \to y \in C = \{\hat{x}^1, \cdots \hat{x}^L\} \subset \mathbb{R}$$

Cell-size $\Delta = \frac{2\text{Amplitude}}{L}$, higher nr. of levels = smaller error
Quantization process can be seen as a coding-decoding process.

$$\xrightarrow{x} \boxed{E_{\theta^i}} \xrightarrow{i} \qquad\qquad \xrightarrow{i} \boxed{D_C} \xrightarrow{\hat{x}}$$

Note: Scalar quantization ≡ we only take a sample at a time, not a vector.

2.1 Scalar Quantizer

A scalar quantizer processes one sample at a time. It assigns the sample to a quantization cell and represents the entire cell using its reconstruction level or corresponding binary index.

2.1.1 Rate

Rate is the amount of coded information spent to represent each source sample. Here it is measured in bits per sample, or in bits per pixel (bpp) when samples are image pixels; for audio and video streams, bitrate is measured in bit/s. Under the ideal fixed-length model with L quantization levels, each index requires

$$R = \log_2 L$$

bits per sample. Fractional values describe an ideal or average rate; an actual fixed-length binary code requires an integer number of bits unless L is a power of two.

2.1.2 Distortion

Distortion is the numerical loss introduced by replacing $x(n)$ with its reconstruction $Q(x(n))$. With squared error, distortion has the squared unit of the signal; averaging it over samples or a probability distribution gives the MSE. We use the MSE to track the error:

$$\text{MSE } d[x(n), Q(x(n))] = |e(n)|^2 = [x(n) - Q(x(n))]^2$$

For signals of duration N:

$$D = \frac{1}{N} \sum_{n=0}^{N-1} d[x(n), Q(x(n))]$$

For random signals:

$$D = \mathbb{E}\left[|X(n) - Q(X(n))|^2\right] = \mathbb{E}\left[|E(n)|^2\right]$$

2.2 Uniform Quantization

A uniform quantizer uses equally spaced thresholds and reconstruction levels. Its single step size Δ controls both precision and rate: smaller steps reduce quantization error but require more levels and therefore more bits. We can use the `floor`,`ceil`,`round`,`fix` operators to perform a quantization, since they map $\mathbb{R} \to \mathbb{Z}$ ($\mathbb{Z}$ mapping).

2.2.1 Definitions

$\forall i$:

- Interval is $(0, A)$ for unsigned data, while $(-\frac{A}{2}, \frac{A}{2})$ for signed data
- Symbol Spacing: $\Delta^i = \Delta = \frac{A}{L}$
- Thresholds: $t^i = t^{i-1} + \Delta$
- Symbols Intervals: $\theta^i = \left(\hat{x}^i - \frac{\Delta}{2}, \hat{x}^i + \frac{\Delta}{2}\right)$
- Symbols: $\hat{x}^i = \left(\frac{t^i + t^{i-1}}{2}\right)$

2.2.2 Unsigned Data

- $i = \left\lceil \frac{x}{\Delta} \right\rceil$
- $\hat{x}^i = i\Delta - \frac{\Delta}{2}$
- $t^i = (i-1)\Delta$
- $Q(x) = \Delta \left\lceil \frac{x}{\Delta} \right\rceil - \frac{\Delta}{2}$

2.2.3 Signed Data (Midtread)

Midtread = '0' is a quantized level, not a threshold

- $i = \text{round}\left(\frac{x}{\Delta}\right)$
- $\hat{x}^i = i\Delta$
- $Q(x) = \Delta \cdot \text{round}\left(\frac{x}{\Delta}\right)$

Alternative case: Midrise = '0' is a threshold, not a level. Bad idea in most of the cases because when there are little oscillations around 0, the quantized value gets amplified.

2.2.4 Dead-zone Quantization for Signed Data

A dead-zone quantizer enlarges the interval mapped to zero. It is useful for sparse transform or prediction residuals because many small coefficients become exactly zero and can then be compressed efficiently by entropy coding. We exploit the midtread quantizer, by extending the '0' region:

$$i = \begin{cases} \text{sign}(x) \left\lfloor \frac{|x| + \frac{\tau\Delta}{2}}{\Delta} \right\rfloor & |x| \geq \tau \\ 0 \text{ otherwise} \end{cases} \qquad x = \begin{cases} \text{sign}(i)\Delta\left(|i| + \frac{1-\tau}{2}\right) & |x| \geq \tau \\ 0 \text{ otherwise} \end{cases}$$

2.2.5 Rate and Distorsion curve

The rate–distortion curve describes the minimum distortion achievable at each coding rate, or equivalently the rate required for a target distortion. It makes the central compression tradeoff explicit: spending more bits generally lowers reconstruction error. We are looking for a relation that allows us to write $D = f(R)$
We use uniform r.v. and the LOTUS[2] expected value of function of r.v., by applying this to uniform quantizers:

$$D = \sigma_Q^2 = \mathbb{E}\left[|X - \hat{X}|^2\right] = \cdots = \frac{1}{A} L \frac{\Delta^3}{12} = \frac{1}{12}\frac{A^2}{L^2} \qquad \text{with } X \sim \mathcal{U}\left(-\frac{A}{2}, \frac{A}{2}\right)$$

[2] Law of the unconscious statistician

Using the fact that $R = \log_2 L$:

$$D = \sigma_Q^2 = \frac{1}{12}\frac{A^2}{L^2} = \sigma_X^2 2^{-2R}$$

Finally we can write the SNR:

$$\text{SNR} \approx 10 \log_{10} \frac{\mathbb{E}[X]}{D} = 10 \log_{10} 2^{2R} \approx 6R$$

SNR is the ratio between signal power and error power, expressed in decibels. The result above means that adding one bit per sample improves SNR by approximately 6 dB under these assumptions.

2.3 Optimal Quantization

An optimal quantizer chooses thresholds and reconstruction levels to minimize an expected distortion for a specified number of levels or rate. "Optimal" always refers to the assumed source distribution and distortion measure. Hypothesis, we use High Resolution Quantization, searching for a locally-optimal solution:

- $L \to \infty \Rightarrow \max_i \Delta_i \to 0$
- $\forall i \ \forall u \in \theta^i : p_x(u) \approx P_i$
- Optimal R-D curve: $\sigma_Q^2 = C_x \sigma_x^2 2^{-2R}$
  Shape Factor $C_x = \frac{1}{12}\left[\int_{\mathbb{R}} P_U^{\frac{1}{3}}(t)dt\right]^3$ and $U = \frac{X}{\sigma_X}$

The shape factor depends only on the PDF *shape*, not on the variance:

$$C_x = 1 \text{ (Uniform)} \qquad C_x = \frac{\sqrt{3}}{2}\pi \approx 2.72 \text{ (Gaussian)}$$

Gaussian signals need $\approx 2.72\times$ more distortion than uniform at the same rate (heavier tails).

2.3.1 High Resolution (HR) Uniform Quantization

High-resolution quantization is an approximation valid when cells are sufficiently small that the source probability density is almost constant inside each cell. It provides simple analytical rate–distortion formulas, but becomes inaccurate at low rates. Hypothesis:

- $L \to +\infty \Rightarrow \Delta \to 0$, X generic r.v.
- $D = \frac{\Delta^2}{12} = \frac{A^2}{12} 2^{-2R}$
- In this case: $\sigma_X^2 \neq \frac{A^2}{12}$

Computing the SNR, we get:

$$\text{SNR} \approx 10 \log_{10} \frac{\mathbb{E}[X]}{D} = 10 \log_{10} \frac{\sigma_X^2}{\frac{A^2}{12} 2^{-2R}} \approx 6R - 10 \log_{10} \frac{\gamma^2}{3} \qquad \text{where } \gamma^2 = \frac{X_{\max}^2}{\sigma_X^2} = \frac{A^2}{4\sigma_X^2}$$

2.3.2 Lloyd-Max algorithm

The Lloyd–Max algorithm is an iterative method for designing a locally MSE-optimal scalar quantizer for a known distribution or training set. It alternates between optimal cell boundaries and optimal reconstruction centroids until the distortion stops decreasing significantly. What if we are not in high resolution hypothesis? Lloyd-Max provides a locally-optimal solution. Steps:

1. Choose an initial dictionary $C^{(0)} = \{\hat{x}_0^i\}_{i=1\cdots L}$ (uniform initialization)
2. Find Best **thresholds** from given dictionary $\Rightarrow$ nearest neighbor method
3. Find Best **dictionary** from given thresholds $\Rightarrow$ centroid condition
4. Iterate 2,3 until convergence.

Nearest Neighbor:

$$k = \arg\min_n |X - \hat{X}^n| \qquad \Longrightarrow \qquad Q(k) = \hat{x}^k \Rightarrow t^i = \frac{\hat{x}^i + \hat{x}^{i+1}}{2} \quad \forall i \in \{1 \cdots (L-1)\}$$

Centroid Condition: computing the gradient of distortion and imposing to zero

$$\nabla D = 0 \qquad \Longrightarrow \qquad \hat{x}^i = \mathbb{E}\left[X | X \in \theta^i\right]$$

Stop condition:

- Threshold model
$$\frac{\sigma^2_{Q,(k-1)} - \sigma^2_{Q,(k)}}{\sigma^2_{Q,(k-1)}} < \epsilon$$
- Reach iteration number $k = K$

2.3.3 Real-life Lloyd-Max adaptation

In real-life we don't have models, but real data:

- $\chi = \{u_1, \cdots, u_m\}$
- $C^{(k)} = \{\hat{x}_0^i\}_{i=1\cdots L}$
- Nearest neighbor rule: $w_k^i = \left\{u_m \in \chi : \forall j \neq i \quad ||u_m - \hat{x}_k^i|| \leq ||u_m - \hat{x}_k^j||\right\}$
- Centroid rule: $\hat{x}_{k+1}^i = \frac{1}{|w_k^i|} \sum_{u_m \in w_k^i} u_m$

This guarantees better performance but not too much improve performances.

2.4 Predictive Scalar Quantization

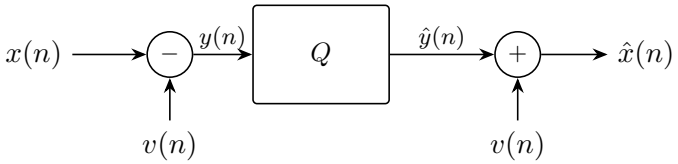
Predictive scalar quantization represents the difference between a sample and its prediction rather than quantizing the sample directly. If prediction is accurate, the residual has lower variance and can reach the same distortion with fewer bits. Idea: exploit sample quantization with prediction.
We want to lower distorsion $D = \sigma_x^2 2^{-2R}$: we have to reduce σ or, to keep the same distorsion, lower the rate.

2.4.1 Sparsification

Sparsification is the concentration of most signal energy or information into a small number of significant coefficients while the remaining coefficients become small or zero. Sparse representations are easier to quantize and entropy-code. Starting from a signal where pixels have the same weight, after applying some sparsification function (Fourier transform for example) some components become more important. Then we make quantization based on human sensitivity.

2.4.2 Prediction

Prediction estimates the current sample from already available samples. The prediction residual $y(n)$ is the part not explained by the predictor and is the quantity sent to the quantizer. We need to encode differences or predicting the differences. How to choose the $v(n)$?
We need a $v(n)$ such that $y(n) = x(n) - v(n)$, then the reconstructed $\hat{y}(n) + v(n) = \hat{x}(n)$.



Basic predictive quantizer: the predictor $v(n)$ is subtracted before the quantizer Q and added back after, so only the (smaller-variance) prediction error is quantized.

2.4.3 Prediction Error

$$q(n) = y(n) - \hat{y}(n) = x(n) - \hat{x}(n) = \bar{q}(n) \qquad \text{the same of the original signal } x$$

2.4.4 Coding Gains

Coding gain measures how much a coding operation reduces distortion at equal rate, or reduces rate at equal distortion, relative to a reference system. Predictive gain G_P is expressed in dB and compares original-signal variance with residual variance. Goal of PQ: minimizing D

$$SNR_p = 10\log_{10}\frac{\sigma_x^2}{D} = G_P + G_Q \qquad \text{where} \begin{cases} G_P = 10\log_{10}\frac{\sigma_x^2}{\sigma_y^2} \text{ predictive gain} \\ G_Q = 10\log_{10}\frac{\sigma_y^2}{D} \end{cases}$$

We are gaining the G_P factor, since $\sigma_x^2 > \sigma_y^2$
We cannot be sure that any prediction is good, we need to look for optimal predictors from previous samples.

2.4.5 Linear Predictors

A linear predictor forms its estimate as a weighted sum of previous samples. Its order P is the number of past samples used; larger orders provide more modeling freedom but increase computation and coefficient-signaling cost. Linear predictors are simple and **optimal for Gaussian r.v.**:

$$v(n) = -\sum_{i=1}^{P} a_i x_{n-i} \qquad \Longrightarrow \qquad y(n) = x(n) - v(n) = \sum_{i=0}^{P} a_i x_{n-i} \quad (a_0 = 1)$$

y is the output of the FIR filter $A(z) = 1 + \sum_{i=1}^{P} a_i z^{-i}$ applied to x. Goal: minimize σ_y^2 over $\vec{a} = [a_1 \cdots a_P]^T$.

2.4.6 Optimal Predictor (Wiener-Hopf)

Expanding the error variance:

$$\sigma_y^2 = \sigma_x^2 + 2\vec{r}^T\vec{a} + \vec{a}^T R_x \vec{a} \qquad \text{where} \begin{cases} \vec{r} = [r_x(1) \cdots r_x(P)]^T \\ (R_x)_{ij} = r_x(|i-j|) \text{ (Toeplitz autocorr. matrix)} \\ r_x(k) = \mathbb{E}\left[X(n)X(n-k)\right] \end{cases}$$

Setting the gradient to zero:

$$\frac{\partial \sigma_y^2}{\partial \vec{a}} = 2\vec{r} + 2R_x\vec{a} = 0 \qquad \Longrightarrow \qquad \vec{a}^{opt} = -R_x^{-1}\vec{r} \qquad \Longrightarrow \qquad \sigma_{y,opt}^2 = \sigma_x^2 + \vec{r}^T\vec{a}^{opt}$$

If we want to use it in predictions, we need the autocorrelation estimation:

$$\hat{r}_x(k) = \frac{1}{N}\sum_{n=0}^{(N-1)-k} X(n)X(n-k)$$

2.4.7 Worked example: AR(1) Gaussian signal

Signal $X(n) \sim \mathcal{N}(0, \sigma^2)$ with autocorrelation $r_x(n-m) = \mathbb{E}\left[X(n)X(m)\right] = \sigma^2\rho^{|n-m|}$.

- **Trivial predictor** $V(n) = X(n-1)$:

$$\sigma_y^2 = \mathbb{E}\left[\left(X(n) - X(n-1)\right)^2\right] = 2\sigma^2 - 2\sigma^2\rho = 2\sigma^2(1-\rho)$$

$$G_P = 10\log_{10}\frac{\sigma^2}{2(1-\rho)\sigma^2} = 10\log_{10}\frac{1}{2(1-\rho)} \qquad \Longrightarrow \qquad G_P > 0 \Longleftrightarrow \rho > \frac{1}{2}$$

  For $\rho = 0.9$: $G_P = 10\log_{10}\frac{1}{0.2} \approx 7$ dB.

- **Optimal predictor of order** $P = 1$: applying Wiener-Hopf with $R_x = \sigma^2$, $\vec{r} = \sigma^2\rho$:

$$a_1^{opt} = -\rho \Rightarrow V(n) = \rho X(n-1) \qquad \sigma_y^2 = \sigma^2 + \sigma^2\rho(-\rho) = \sigma^2(1-\rho^2)$$

$$G_P = 10\log_{10}\frac{1}{1-\rho^2} \geq 0 \quad \forall\rho \qquad \text{(never worse than no prediction)}$$

  Comparison: $1-\rho^2 = (1-\rho)(1+\rho) < 2(1-\rho)$ for $\rho < 1 \Rightarrow$ optimal always beats trivial.

- **Optimal predictor of order** $P = 2$: now $R_x = \sigma^2 \begin{bmatrix} 1 & \rho \\ \rho & 1 \end{bmatrix}$, $\vec{r} = \sigma^2[\rho,\ \rho^2]^T$:

$$\vec{a}^{opt} = -R_x^{-1}\vec{r} = -\frac{1}{1-\rho^2}\begin{bmatrix} 1 & -\rho \\ -\rho & 1 \end{bmatrix}\begin{bmatrix} \rho \\ \rho^2 \end{bmatrix} = \begin{bmatrix} -\rho \\ 0 \end{bmatrix}$$

  Same solution as $P = 1$: the second tap is useless because in an AR(1) source all the information about $X(n)$ is contained in $X(n-1)$ (Markov property). Increasing the order brings no further gain.

2.4.8 Prediction Gain: filter order impact

- Order increase = gain improve
- Complexity vs Performance tradeoff

2.4.9 Local adaptation

Some images are not stationary, we need to split them into omogeneous blocks and compute different predictors:

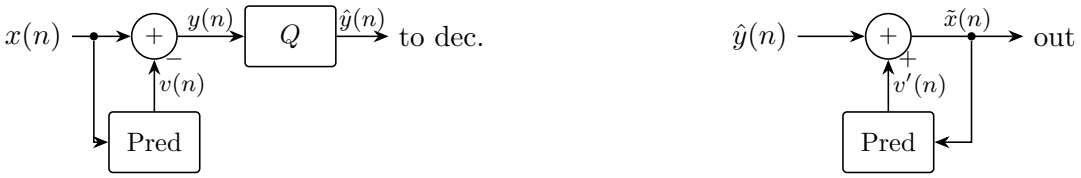
- Large Blocks: small overhead but no local features
- Small Blocks: excellent tracking of features, but high overhead
- Rate increase by $\frac{N \cdot B}{m^2}$ where N = bits*pixel/block, B number of blocks, M^2 number of blocks

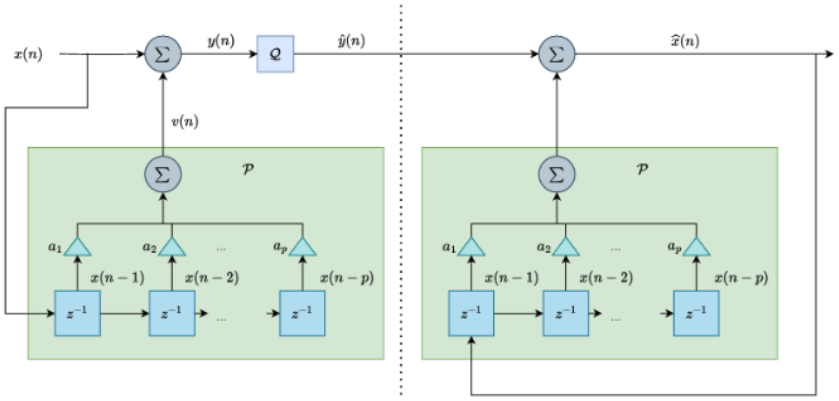
If we split an image in too many blocks, we lose efficiency. For low bitrates is better to use large blocks.

2.4.10 Wrong scheme: open-loop prediction (drift)

Naive idea: encoder predicts from the *original* samples $x(n)$.

ENCODER **DECODER**





WRONG! The predictor operator is the same at the encoder and decoder but the input is different: $x \neq \tilde{x}$! This causes **drift**.

Problem: the encoder predictor uses the original $x(n)$, but the decoder can only use the reconstructed $\tilde{x}(n)$. Since $x \neq \tilde{x}$, the two predictions $v(n) \neq v'(n)$ diverge and the error accumulates over time: this is the **drift** problem.
Numerical example (3-bit quantizer, levels $\{-9, -6, -3, 0, 3, 6, 9\}$), predictor = previous sample:

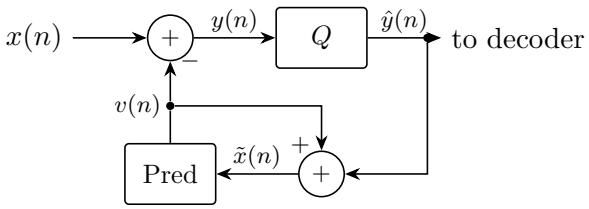
step	original	ENC pred	ENC error	quantized	DEC pred	reconstr.
1	10	–	–	9	–	9
2	11	10	1	0	9	9
3	12	11	1	0	9	9
4	13	12	1	0	9	9
5	14	13	1	0	9	9
6	18	14	4	3	9	12

Encoder predicts from originals (10,11,12,...), decoder is stuck at 9: drift grows unbounded.

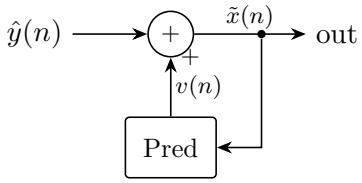
2.4.11 Correct scheme: closed-loop prediction (DPCM)

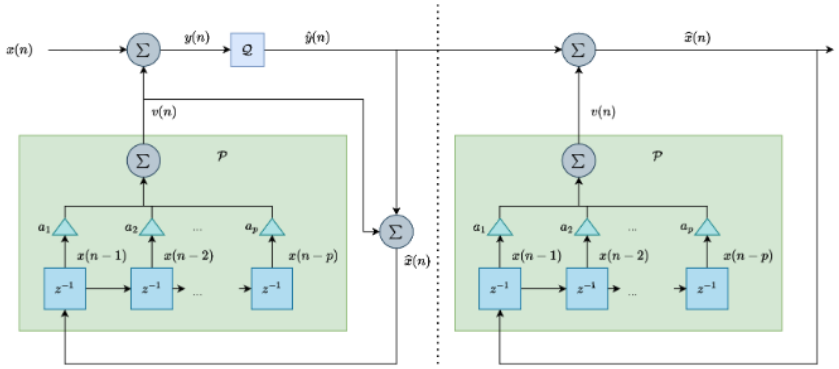
DPCM (Differential Pulse-Code Modulation) is predictive quantization with matching reconstruction loops at encoder and decoder. Both sides predict from previously reconstructed samples, preventing predictor mismatch and unbounded drift. **Solution**: the encoder embeds a copy of the decoder, and predicts from the *quantized/reconstructed* samples $\tilde{x}(n)$ — exactly the same data the decoder has.

ENCODER



DECODER





This is the correct scheme: the predictor is fed with the same data (quantized past samples) both at the encoder and the decoder.

Comment on the blocks:

- Encoder: prediction error $y(n) = x(n) - v(n)$ is quantized to $\hat{y}(n)$ and sent.
- Reconstruction loop (inside the encoder): $\tilde{x}(n) = \hat{y}(n) + v(n)$, stored in a buffer of past quantized samples; the predictor reads only from this buffer.
- Decoder: computes the same $v(n)$ from the same $\tilde{x}$ buffer, then $\tilde{x}(n) = \hat{y}(n) + v(n)$.
- Encoder and decoder stay **synchronized**: same inputs to the predictor on both sides ⇒ no drift. The quantization error on x equals the one on y (see Prediction Error above).

Same numerical example with the correct scheme:

step	original	ENC pred	ENC error	quantized	DEC pred	reconstr.
1	10	–	–	9	–	9
2	11	9	2	3	9	12
3	12	12	0	0	12	12
4	13	12	1	0	12	12
5	14	12	2	3	12	15
6	18	15	3	3	15	18

Encoder and decoder predictions always coincide: error stays bounded, no drift.

2.4.12 Impact of entropy coding on PQ

Without entropy coding ($R = \log_2 L$) PQ looks disappointing: that rate estimate is **too pessimistic**, because most quantization indexes are zero (e.g. $L = 19$ levels ⇒≈ 84% of indexes are 0). A variable-length code (short codeword for index 0, longer for rare indexes) reveals the true gain:

- ΔPSNR ≈ +20 dB at 1 bpp vs direct UQ
- equivalently −86% rate at 30 dB target quality

PQ works only if: (1) prediction computed on **quantized** data, (2) signal sufficiently correlated ($\rho > 1/2$ for trivial first-order prediction), (3) followed by **entropy coding**.

2.4.13 Direct vs Predictive Quantization

With predictive I still have to use a lot of bits to represent information. Idea: why should we use fixed-length coding? Using PQ + Entropy coding we outperform direct quantization, since quantization indexes are Gaussian distributed. At 1bpp we have ΔPSNR = 20dB.

3 Lossless Coding

Lossless coding maps source data to a reversible bitstream: decoding reproduces every original symbol exactly. Compression comes only from statistical structure and repeated patterns, not from discarding information.

3.1 Definitions

- **Alphabet**: $\mathcal{X} = \{x_1, \cdots, x_M\}$ Symbols to encode
- **Code**: Application $\mathcal{C} : \mathcal{X} \to \{0,1\}^\star$ set of finite-length bit strings, can be Fixed or Variable length.

3.2 Code Types

3.2.1 Fixed-Length Code

A fixed-length code assigns the same number of bits to every source symbol. It offers simple parsing and constant per-symbol rate, but cannot assign shorter descriptions to frequent symbols. Good: straight forward parsing of the codewords.
Bad: Very large rate $R = \log_2 L$, we are assuming all symbols are equally probable.
Needed bits: $\lceil \log_2 M \rceil$

3.2.2 Variable-Length Code

A variable-length code assigns different codeword lengths to different symbols. Its rate is the expected codeword length $\bar{L}$, measured in bits per source symbol, so frequent symbols should receive shorter codewords.

- Statistical Approach: we make use of source entropy $H(x)$, to build an optimal Huffman code. We could use also suboptimal aritmethic codes, which scale very well.
- Euristic Approach: if we don't know data statistics, universal coding (LZW)

We define l_i as the length of the codeword c_i.
We need also two conditions:

- Decodability condition / prefix condition (each word is not a prefix for other words)
- Non-equiprobable symbols

How to write this in math terms?

$$f : x_i \to c_i \qquad \text{avg. length } \bar{L} = \sum p_i l_i$$

We want to minimize the average length.

3.3 Variable-length codes theorems

3.3.1 McMillan's Theorem

Decodable codes do not improve performance w.r.t. instantaneous/prefix codes.
The best possible prefix code = the best possible decodable code. We have to focus on prefix codes.

3.3.2 Kraft's Inequality

$$\sum_i 2^{-l_i} \leq 1 \qquad \iff \qquad \exists \text{ instantaneous code with lengths } \{l_1, \cdots, l_M\}$$

If the equality is verified, we say the code is **complete**, otherwise we can try to push more from the code.
We can associate a binary code to a binary tree.

3.3.3 Proof for Kraft's ⇒ (necessity)

Build a binary tree with $L_{\max} = \max(l_i)$ depth.
We remove nodes which have prefix equal to a codeword, then each removed subset is disjoint from the others:

$$\text{Removed nodes } : \sum_i 2^{L_{\max} - l_i} \leq 2^{L_{\max}}$$

3.3.4 Proof for Kraft's ⇐ (sufficiency)

Greedy Construction Algorithm:

- Sort Length $l_1 \leq \cdots \leq l_N$
- $\forall$ step k: pick available node at depth l_k
- Mark descendants as forbidden

Finally: there is no way to improve it.

3.4 Recall: Information Theory

3.4.1 Information

Self-information measures how surprising one event is. It is measured in bits when the logarithm has base two: rare events carry more information than probable events.

$$I(x_i) = -\log_2 p(x_i) = -\log_2 p_i$$

Joint Information:

$$I(x_i, x_j) \leq I(x_i) + I(x_j)$$

Equality only if symbols are independent.

3.4.2 Source Entropy

Source entropy is the average self-information generated per symbol by a memoryless source. It is measured in bits per symbol and is the fundamental lower bound approached by lossless codes matched to the source statistics.

$$H(X) = \sum_i p_i I(x_i) = -\sum_i p_i \log_2 p_i$$

3.4.3 Max Entropy Distribution

We want $p^\star$ such that:

$$p^\star = \arg\max_p \sum_{i=1}^{M} p_i H(x_i) \qquad \text{with } \sum_{i=1}^{M} p_i = 1$$

Using Lagrange Multipliers:

- we consider a function $f : \mathbb{R}^n \to \mathbb{R}$
- then we build a function $J(x, \lambda) = f(x) + \lambda \cdot \varphi(x)$
- then we set $\forall i$ the $\nabla J(x_i, \lambda) = 0$

In this case:

- $f(p) = -\sum_{i=1}^{M} p_i \log_2(p_i)$
- $\varphi(p) = \sum_{i=1}^{M} p_i - 1$

Thus the gradient is:

$$\frac{\partial J}{\partial x_i} = -\left(\log_2 e + \log_2 p_i\right) + \lambda \qquad \frac{\partial J}{\partial p_i}(p^\star) = \varphi(p) = 0$$

By imposing =0 to the x_i component of gradient:

$$\log_2 p_i^\star = \lambda - \log_2 e$$

3.4.4 Joint Entropy

Joint entropy is the average information needed to describe two random variables together. It includes both their individual uncertainty and any statistical dependence between them.

$$H(X,Y) = -\sum_{i,j} p_{ij} \log p_{ij}$$

3.4.5 Conditional Entropy

Conditional entropy $H(X|Y)$ is the uncertainty that remains about X after Y is known. In coding terms, it is the ideal average rate for coding X when the decoder also has Y as side information.

$$H(X|Y) = \sum_j p_j H(X|Y = Y_j)$$

Using chain rule:

$$H(X,Y) = H(Y) + H(X|Y) = H(X) + H(Y|X)$$

3.5 Optimal Code

An optimal lossless code minimizes expected codeword length for the source probability distribution while remaining uniquely decodable. Its performance is compared with entropy in bits per source symbol.

3.5.1 Math Formulation

- $\forall i = 1 \cdots M,\ l_i \in \mathbb{N}$ satisfies Kraft Inequality
- Average length $\mathcal{L} = \sum_{i=1}^{M} p_i l_i$

Constrained Minimization Problem to find the optimal length $\mathcal{L}$ (it is a lowerbound):

$$\vec{l}^\star = \arg\min_{\vec{l}} \sum_i p_i l_i \qquad \text{subject to Kraft: } \sum_i 2^{-l_i} = 1$$

Appyling the Lagrangian Multipliers method we end up with:

$$p_i = 2^{-l_i^\star} \qquad \Longrightarrow \qquad \mathcal{L}^\star = -\sum_i p_i \log_2 p_i = H(x)$$

3.5.2 Shannon's source coding theorem

Shannon's source coding theorem states that entropy is the fundamental average-rate limit for lossless compression. A practical symbol code may stay less than one bit per symbol above this limit, while coding long blocks can make the overhead per symbol arbitrarily small.

$$\forall i \in \{1 \cdots M\} \qquad \exists k \in \mathbb{N} \qquad : \qquad H(x) \leq \mathcal{L}^\star \leq \mathcal{L} < H(x) + 1$$

Upper band proof: using $l_i = \lceil -log_2 p_i \rceil$, we prove that satisfies Kraft's (having $\delta_i \in \{0, 1\}$, $\epsilon_i = 2^{\delta_i}$)

$$l_i = -\log_2 p_i + \delta_i \qquad \Longrightarrow \qquad 2^{-l_i} = p_i 2^{\delta_i} \qquad \Longrightarrow \qquad \sum_i 2^{-l_i} = \sum_i p_i \epsilon_i \leq \sum_i p_i = 1$$

Average length proof

$$l_i = \lceil -log_2 p_i \rceil < -log_2 p_i + 1 \qquad \Longrightarrow \qquad \sum_i p_i l_i < \sum_i (-p_i \log_2 p_i + p_i) \Longrightarrow \mathcal{L} < H(x) + 1$$

3.5.3 Huffman Coding

Huffman coding is a prefix-code construction that gives shorter binary codewords to more probable symbols. It minimizes average length among symbol-by-symbol binary prefix codes with integer codeword lengths.

1. Build leaves nodes with respective probability
2. Take the two least probable nodes
3. Merge these nodes, keeping the sum of the probabilities
4. Re-sort all the nodes in descending order of probability
5. Iterate these steps (from 2.) until we reach a single node with $p = 1$
6. Assign at each branch either 0 or 1.

Using only PQ + Huffman Coding we cannot reach < 1bpp coding rate, how can we do better?

3.5.4 Block Coding

Block coding treats K consecutive source symbols as one compound symbol. The code's rate is total block length divided by K, measured in bits per original symbol; larger blocks reduce integer-length overhead but make the alphabet and probability model much larger. We take X^K as blocks of K symbols: we also know that $H(X^K) \leq \mathcal{L}^\star < H(X^K) + 1$ for Shannon, by scaling:

$$H(X^K) \leq \mathcal{L}^\star < H(X^K) + 1 \qquad \Longrightarrow \qquad \frac{H(X^K)}{K} \leq \frac{\mathcal{L}^\star}{K} < \frac{H(X^K)}{K} + \frac{1}{K}$$

We can make $\frac{1}{K} \to 0$ if we grow block size to $K \to \infty$.

$$\text{Avg. length per symbol } \mathcal{L}_s^\star \approx \frac{H(X^K)}{K} \leq H(x)$$

We define now the Entropic Rate of the source (ultimate bound for lossless coding):

$$\lim_{k \to \infty} \frac{H(X^k)}{k} = \mathcal{H}(X)$$

Entropy rate $\mathcal{H}(X)$ is the average new information produced by each symbol of a source with memory. It can be lower than single-symbol entropy because past symbols help predict the next one.

3.5.5 Limits of Huffman

- Complexity **exponential** in K: block alphabet has M^K words
- Joint probability estimation for large blocks is costly and unreliable
- Cannot handle $H(X) < 1$ efficiently symbol by symbol (min 1 bit/symbol)

Arithmetic coding solves at least the complexity problem: **block coding with linear complexity** $O(n)$, encoding the whole sequence as a single interval in $[0, 1)$. Penalty only $+2$ bits *per block*, vs Huffman's $+1$ bit *per symbol*.

3.5.6 Arithmetic coding

Arithmetic coding represents an entire symbol sequence with one fractional interval whose width equals the sequence probability. Its average rate can approach entropy without requiring an exponentially large block-code dictionary. The alphabet is M long, we take a K block. Huffman Alphabet has M^K possible words.
Since complexity increases with growing K, we take a suboptimal code (excluding non-sensed words).
IDEA:

- Take a squence
- Each symbol $\in [0, 1]$
- For each new symbol use 2 multiplications and 2 sums to update interval.
- We encode each symbol with a number (the center of the probability interval $c_i = c_{i-1} + \frac{p_i - p_{i-1}}{2}$)

How many bits do we need?

$$\lceil -\log_2 p_i \rceil + 1 < -\log_2 p_i + 2$$

So basically we have:

- Length:
$$L(n) = -\left\lceil \sum_{i=1}^{n} \log_2 p(x_i) \right\rceil + 1 = E[\bar{L}(n)]$$
- Average Length:
$$\bar{L}(n) = \frac{-\sum_{i=1}^{n} \log_2 p(x_i) + 2}{n}$$
- Precision:
$$\frac{\prod_{i=1}^{n} p_i}{2}$$

We have that

$$\mathcal{L} < H(x) + \frac{2}{n} \qquad \overset{n\to\infty}{\longrightarrow} \qquad H(x)$$

It is a context based encoding, it uses conditional probabilities.

3.5.7 Adaptivity and Context-based coding

Adaptive coding updates probability estimates while processing the stream. Context-based coding conditions those estimates on already decoded neighboring symbols, allowing encoder and decoder to exploit source memory without transmitting the context itself.

- **Adaptivity**: symbol statistics learned during encoding via occurrence counts, updated at both encoder and decoder with an agreed rule ⇒ handles non-stationary sources.
- **Context-based**: instead of estimating $P(X^K)$ directly, condition on the N_S previous symbols ($N_C = M^{N_S}$ contexts $\equiv N_C$ arithmetic encoders switching among each other). Reaches the entropy rate $\mathcal{H}$ without massive block sizes.
- Context design rule: too large → sparse data, unreliable estimation; too small → misses dependencies.

3.5.8 Why Arithmetic is preferred over Huffman (wrap-up)

Advantages: linear complexity $O(n)$; exploits high-order dependencies removing the non-dyadic penalty; context coding models high-order statistics cheaply; adaptive.
Disadvantages: tricky implementation (precision, carry propagation); needs initialization; context selection non-trivial; adaptivity needs large training data.

3.5.9 Context-based coding

Context-based coding predicts the probability of the next symbol from a causal neighborhood already known to both encoder and decoder. Better contexts lower coding rate when they make the next symbol more predictable, but excessively large contexts produce unreliable statistics. It is based on context (near pixels), so joint probability.
How to choose the context? Look the data, or use learning-based techniques to learn prob. distribution.

3.6 Other Techniques

3.6.1 Unsigned Exp-Golomb

Unsigned Exp-Golomb is a universal variable-length code for non-negative integers. Small values receive short codewords, so it is effective for syntax elements and prediction residual magnitudes concentrated near zero.

Encoder:

- start with $n = 0$
- $\forall n \in \{1, 2, \cdots\}$ we write $n + 1$ in binary
- we compute $b = \lfloor \log_2(n + 1) \rfloor + 1$
- we add $b - 1$ zeros in front of the binary number

Decoder:

- if next bit is 1 → set to 0
- count how leading zeros $(b - 1)$ we
- we read the b-sized bitstring
- we get the decimal representation a tract 1

3.6.2 Signed Exp-Golomb

Signed Exp-Golomb first maps positive and negative integers to non-negative integers, then applies unsigned Exp-Golomb coding. It preserves short codewords around zero without needing a separate sign bit.

Encoder:

- $m(n) = \begin{cases} 2n - 1 & \text{if } n > 0 \\ -2n & \text{if } n \leq 0 \end{cases}$
- encode $m(n)$ with previous method

Decoder:

- decode with unsigned exp-golomb
- remap $n = \begin{cases} \frac{m+1}{2} & \text{if odd} \\ -\frac{m}{2} & \text{if even} \end{cases}$

3.6.3 Dictionary-based coding

Dictionary-based coding replaces repeated symbol strings with references to entries in a shared dictionary. Its rate improves when long patterns recur, because one dictionary index can represent many source symbols. **Asymptotic Optimality Theorem**:
If stationary and ergodic source, Dictionary-based coding is asymptotically optimal.
Examples of Dictionary-based coding: LZ[3]77, LZ78, LZW[4] (used in Deflate, GIF, ZIP), LZMA.

3.6.4 LZW coding

LZW is an adaptive dictionary code in which encoder and decoder construct the same phrase dictionary while processing the stream. Only dictionary indexes are transmitted; the dictionary itself need not be sent.

Encoder and Decoder - Greedy Matching Process + Dictionary Evolution and Merging:

- read inputstream symbol by symbol
- foreach new input, check if (prefix W + next K) exists in the dictionary
- if $\exists\ (W + K)$, read next symbol until new pattern.
- if $\nexists\ (W + K)$, index of longest known W is sent to bitstream, $W + K$ added to dictionary
- then prefix merging and update local statistics

3.6.5 Examples of real-life Coding

- JBIG-1: template of 10 pixels, progressive coding (context-based arithmetic coding).
- JBIG-2: images classified as text, halftone or other, it is used in PDFs, (context-based arithmetic coding).
- JPEG-LS: Lossless image compression, encoding[5] of $e = x - \hat{x}$, where $\hat{x} = \begin{cases} \min(A, B) & C \geq \max(A, B) \\ \max(A, B) & C \leq \min(A, B) \\ A + B - C & \text{otherwise} \end{cases}$
- PNG: uses LZ77 + Huffman, not so difficult, can use prediction
- DPCM[6]: 1D/2D spatial prediction, encodes error $y = x(n) - p(n) = x(n) - x(n-1)$ (zero-centered)

3.6.6 Neural Lossless Coding (NLC) Techniques

Neural lossless coding uses a neural probability model to estimate symbol distributions, then applies entropy coding without changing source values. Cross-entropy $H(P, Q)$ is the achieved ideal rate under model Q when true data follow P; the KL term measures extra bits caused by model mismatch.

$$H(P, Q) = H(P) + D_{KL}(P \parallel Q)$$

It takes into account source entropy $H(P)$ and penalty for model inaccuracy based on Kullback-Leibler distance.
Models:

[3] Lempel-Ziv
[4] Lempel-Ziv-Welsch
[5] using Exp-Golomb
[6] Differential Pulse Coding Modulation

- Autoregressive models: causal density estimation
- Latent Variable Models: mapping to manifolds
- Neural Predictive Coding (Non-linear prediction), uses multi-layer Perceptron

$$\hat{y} = f_{MLP}(S, W) \rightarrow \text{INPUT S: local causal neighborhood}$$

NLC has an high complexity, is more efficient for high-resolution images. We also need to encode Biases and Weights, still convenient for small NNs but we also need HW requirements = complexity grows.

3.6.7 Guidelines to reach $\mathcal{L} \approx \mathcal{H}$

- For Memoryless: use Huffman/Arithmetic
- For stationary with memory: use Context-adaptive (Markov Chains)
- For Locally Stationary: use adaptive
- For Highly Complex: use Neural Models

Dictionary is the best if I have to compress text.

4 Transform Coding and JPEG

4.1 Introduction with Block Coding

Block coding groups M source samples into a vector and allocates bits among its components. R_k is the rate assigned to component k, measured in bits per vector component, while $R_{tot} = \sum_k R_k$ is the available bit budget for the whole vector. Take a $X = [x_1 \cdots x_M]^T$ random vector as source of information of size M.
We suppose $\forall$ component to know the $\sigma_k^2 = \text{var}(x_k)$. The distorsion of the k-th element then is $D_k = c_k \sigma_k^2 2^{-2R_k}$.
Globally, across all vector, we have:

$$D = \frac{1}{M} E\left[||X - Q(X)||^2\right] = \cdots = \frac{1}{M} \sum_{k=0}^{M-1} D_k = \frac{1}{M} \sum_{k=0}^{M-1} c_k \sigma_k^2 2^{-2R_k}$$

4.2 Huang-Schulteiss (HS) formula

The Huang–Schultheiss formula is an optimal continuous bit-allocation rule under a high-resolution rate–distortion model. It distributes a fixed total rate so that high-variance or expensive components receive more bits and all active components end with equal distortion. The optimal allocation is found by minimizing distortion D over the rate vector $\vec{R}$ under a total-rate constraint:

$$\min D(K) = \frac{1}{M} \sum_{k=0}^{M-1} D = \frac{1}{M} \sum_{k=0}^{M-1} c_k \sigma_k^2 2^{-2R_k} \qquad \text{s.t.} \quad \sum_{k=0}^{M-1} R_k \leq R_{tot}$$

4.2.1 HS formula derivation

Build the Lagrangian of the constrained problem:

$$J(\vec{R}, \lambda) = \frac{1}{M} \sum_{k=0}^{M-1} c_k \sigma_k^2 2^{-2R_k} + \lambda \left(\sum_{k=0}^{M-1} R_k - R_{tot} \right)$$

Setting $\frac{\partial J}{\partial R_k} = 0$:

$$2^{-2R_k^\star} = \frac{M\lambda}{2\ln 2} \cdot \frac{1}{c_k \sigma_k^2} \qquad \Longrightarrow \qquad R_k^\star = \lambda' + \frac{1}{2}\log_2\left(c_k\sigma_k^2\right)$$

Imposing the constraint $\sum_{k=0}^{M-1} R_k^\star = R_{tot}$ determines $\lambda' = \frac{R_{tot}}{M} - \frac{1}{2}\log_2\left(c_{GM}\sigma_{GM}^2\right)$, so:

Huang-Schulteiss' Formula: $R_k^\star = \dfrac{R_{tot}}{M} + \dfrac{1}{2}\log_2\left[\dfrac{c_k\sigma_k^2}{c_{GM}\sigma_{GM}^2}\right]$ with $\begin{cases} c_{GM} = \sqrt[M]{\prod_{k=0}^{M-1} c_k} \\ \sigma_{GM}^2 = \sqrt[M]{\prod_{k=0}^{M-1} \sigma_k^2} \\ GM = \text{Geometrical Mean} \end{cases}$

Reading: start from uniform allocation $\bar{R} = \frac{R_{tot}}{M}$, then components with variance above the geometric mean get more bits, those below get fewer.

4.2.2 HS formula interpretation

Recall:

- Arithmetic Mean: $Z_{AM} = \frac{1}{M}\sum_{i=0}^{M-1} z_i$
- Geometric Mean: $Z_{GM} = \sqrt[M]{\prod_{i=0}^{M-1} z_i}$
- For Jensen Inequality $Z_{GM} \leq Z_{AM}$

We are looking for a uniform resource distribution $\bar{R} = \frac{R_{tot}}{M}$, so the k-th component distorsion using $R_k^\star$ is:

$$D_k^\star = c_{GM}\sigma_{GM}^2 2^{-2\bar{R}} \qquad \Longrightarrow \qquad D^\star = D_k^\star$$

At the optimum **all components have the same distortion**.
In the Gaussian case:

$$D^\star = D_k^\star = c_{\mathcal{N}}\sigma_{GM}^2 2^{-2\bar{R}}$$

4.2.3 Why the Geometric Mean is the key quantity

$D^\star = c_{GM}\sigma_{GM}^2 2^{-2\bar{R}}$: at fixed total rate, the R-D performance depends *only* on the geometric mean of the variances. By Jensen $\sigma_{GM}^2 \leq \sigma_{AM}^2$ with equality iff all σ_k^2 are equal.
Identically distributed components ($\sigma_k^2 = \sigma_X^2$, $c_k = c_X$ $\forall k$):

$$R_k^\star = \bar{R} \qquad D^\star = c_X\sigma_X^2 2^{-2\bar{R}} = D_{PCM}$$

Block coding brings **no improvement** over sample-by-sample coding in this case. The signal must be *sparse* (diverse variances) to benefit: the whole strategy of transform coding is to reduce σ_{GM}^2 while keeping energy constant.

4.3 Transform Coding

Transform coding converts correlated source samples into coefficients in another basis, quantizes those coefficients, and transforms them back during decoding. Compression improves when the transform concentrates energy into few coefficients and enables unequal bit allocation. IDEA: can we modify our signal where symbols have different distribution (sparse signal)?
We use **Linear Transforms** (change in base of the vectors, like rotations) to reach **signal sparsification** (few big samples with many lower samples). We are looking for a transform T which is:

- Reversible $Y = T(X) \Longleftrightarrow X = T^{-1}(Y)$

- Input data is of size M
- Input X is not sparse, while output is sparse Y
- Quantization error of Y is the same of the X one

An example could be the Fourier Transform.

4.3.1 Orthogonal Transforms

An orthogonal transform is an energy-preserving linear change of basis whose inverse equals its transpose. This property makes reconstruction simple and ensures that squared error is unchanged by moving between source and transform domains. Recalling Orthogonal Matrix property: $T^{-1} = T^T$, we have that $Y = TX$ and $X = T^T Y$
Orthogonal Transforms Isometry property also guarantees that $||Y||^2 = ||TX||^2 = ||X||^2$ and $D_X = \cdots = D_Y$

4.3.2 Orthogonal Transforms applied to Block Coding

Assume we have X random vector of M components, $X \sim \mathcal{N}(0, \sigma_x^2)\ \forall k$, then:

$$\text{PCM Distorsion: } D_{PCM} = c_{\mathcal{N}} \sigma_x^2 2^{-2\bar{R}}$$

Let's use a generic O.T.:

$$T : \begin{cases} Y = TX \\ D_X = D_Y \end{cases} \qquad \Longrightarrow \qquad D_Y = c_{GM,Y} \sigma_{GM,Y}^2 2^{-2\bar{R}}$$

If X, Y are gaussian, we can also use $c_{GM,Y} = c_{GM,X} = c_{\mathcal{N}}$, so for any $T \to \sigma_{AM,Y}^2 = \sigma_{AM,X}^2 = \sigma_X^2$, and also:

$$\text{Coding Gain: } G_T = \frac{D_{PCM}}{D_Y} = \frac{\sigma_{AM,Y}^2}{\sigma_{GM,Y}^2} = \frac{\sigma_X^2}{\sigma_{GM,Y}^2} \qquad \Longrightarrow \text{Jensen's Ineq.} \to G_T \geq 1$$

Transform coding gain is the distortion ratio between direct sample coding and transform coding at equal rate. It is dimensionless as a ratio and is often reported in dB as $10 \log_{10} G_T$.

4.3.3 Transform Coding example

We have a 2D random vector constrained in S, we know joint probability:

$$f_{x_1,x_2} = \begin{cases} \frac{1}{\Delta_1 \Delta_2} & (x_1, x_2) \in S \\ 0 & \text{otherwise} \end{cases} \qquad \text{where } \Delta_1 \gg \Delta_2 \text{ and } x_1 \sim x_2 \sim \mathcal{U}\left[-\frac{\Delta_1}{2\sqrt{2}}, \frac{\Delta_1}{2\sqrt{2}}\right]$$

By applying the rotation, x_1 and x_2 become independent:

$$\vec{Y} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix} \vec{X} \qquad \Longrightarrow \qquad D_2 \ll \sigma^2 \Longrightarrow D = D_1$$

4.4 Practical Algorithms for Resource Allocation

Resource allocation selects how many bits each coefficient or subband receives under a total-rate constraint. An allocation is good when moving one additional bit to any other active component would not reduce total distortion further.

4.4.1 Greedy Algorithm

Requires nr. of iterations $\approx$ nr. of bits, steps:

- Start with 0 bits
- Take largest distorsion and assign 1 bit
- Recompute distorsions
- Iterate until we finish the available bits

4.4.2 Modified HS algorithm

Quicker than the Greedy Algorithm, steps:

- Compute $R_K^\star$ with HS
- If some $R_K^\star$ are negative, remove concerned components and relative variances (zero bit coded)
- Repeat until there are no more negative Rs
- Remove all decimal parts from the obtained values
- Reallocate residual bits to bigger Rs

4.5 Towards Optimal Transform

4.5.1 Karhunen-Loève Transform (KLT)

The Karhunen–Loève Transform is the data-dependent orthogonal transform formed from covariance-matrix eigenvectors. It decorrelates transform coefficients and is optimal for energy compaction under its statistical model. Assuming we have X: zero mean random vector of size M
Such that $R_X = E[XX^T]$, R_X has M orthogonal eigenvectors $u_1 \cdots u_M$
Then KLT $\equiv$ an orthogonal Matrix T_{KLT} where the rows are the eigenvectors:

$$T_{KLT} = \left[u_1 \vdots u_2 \vdots \cdots \vdots u_M\right]^T$$

Properties:

- $T_{KLT}^{-1} = T_{KLT}^T$
- Decorrelating transofrm, $E[y_i y_j] = \lambda_i \delta_{ij}$
- Best energy correlation (sparsity) $\forall N \in \{1 \cdots M\} : \sum E[y_i^2] \geq \sum E[z_i^2]$

3D intuition: take the spatial direction of point distribution and rotate it to the max,medium,min power axis. After transform we have sparsity and size reduction (Similar to ML PCA = Principal Component Analysis).
In the case of Gaussian data, this transform minimizes the geometric mean.
Example of usage: Multi/Hyper spectral imaging, where there is a large correlation across spectral bands.
KLT is not used typically in image codecs, due to:

- Computational Complexity $\propto O(N^3)$ for eigenvectors, $O(N^2)$ for matrix multiplications
- Signaling overhead, T_{KLT} sent apart
- Model Correctness: images are locally stationary

4.6 Frequency Transforms

4.6.1 1D-DFT for Compression

The Discrete Fourier Transform (DFT) represents a finite sequence as complex sinusoidal frequency components. Each coefficient measures amplitude and phase at one discrete frequency.

$$y[k] = \frac{1}{\sqrt{M}} \sum_{n=1}^{M} x[n] \exp\left(-j\frac{2\pi}{M}kn\right) \qquad \forall k = \{1 \cdots M\}$$

It is the same as a T_{DFT} transform where:

$$T_{DFT} = \frac{1}{\sqrt{M}} \left[W_m^{i \cdot j}\right] \qquad \text{where} \begin{cases} i \in \{0 \cdots M-1\} \\ j \in \{0 \cdots M-1\} \\ W_m = \exp\left(-j\frac{2\pi}{M}\right) \end{cases}$$

4.6.2 2D-DFT for Compression

$$y = Ax \qquad \text{where } A \text{ has size } N^2 \times N^2$$

4.6.3 DFT Separable Implementation

$$Y = (TX)\,T^T \qquad \text{with 2D basis func. } B_{kl}(n,m) = \frac{1}{N} \exp\left(j\frac{2\pi}{N}(kn+lm)\right)$$

First product TX analyses the col. (V) frequency variation, while product with T^T makes the row (H) analysis.
By sampling and periodizing the signal ⇒ big jumps in the signal at period start/end ⇒ **Frequency Leakage**.
In Frequency Leakage some energy leaks to the high frequencies (due to the big jumps = high variation).
Plotted DFT has a cross-like structure, usually it is plotted the $\log_{10}|DFT|$ version of the data
IDEA: why don't we put a mirror on the signal instead of periodizing, s.t. we reduce big jumps?

4.6.4 DCT Transform Matrix

The Discrete Cosine Transform (DCT) represents a finite real sequence using real cosine basis functions. Its symmetric boundary model reduces frequency leakage and usually compacts image energy better than a directly periodized DFT. Steps to obtain it:

- Create a mirrored-periodic version of $x \to x_{sym}$
- DFT of x_{sym}
- Apply frequency domain modulation (to obtain real coefficients)
- Keep only M coefficients

4.6.5 1D-DCT Transform

$$T_{DCT} = \forall n = \{0 \cdots M-1\} \to \begin{cases} \frac{1}{\sqrt{M}} & k = 0 \\ \sqrt{\frac{2}{M}} \cos\frac{(2n+1)k\pi}{2M} & k > 0 \end{cases}$$

4.6.6 2D-DCT Transform: Block-Based DCT

We just apply the DCT on both the horizontal and vertical axis.
Usually we apply an $n \times n$ block-based DCT transform, so with 64 vector of the basis.
The $n \times n$ matrix represents no variations with the UL corner, total horizontal variation on the UR corner, total vertical variation on the BL corner and a combination of total H/V variation in the BR corner.
Using the Block-Based DCT we get a very sparse signal, it is good because we apply the transform on blocks of the image: blocks are locally stationary, if there are more informations we use more bits, otherwise no.
How to decide the quantization step of each coefficient?

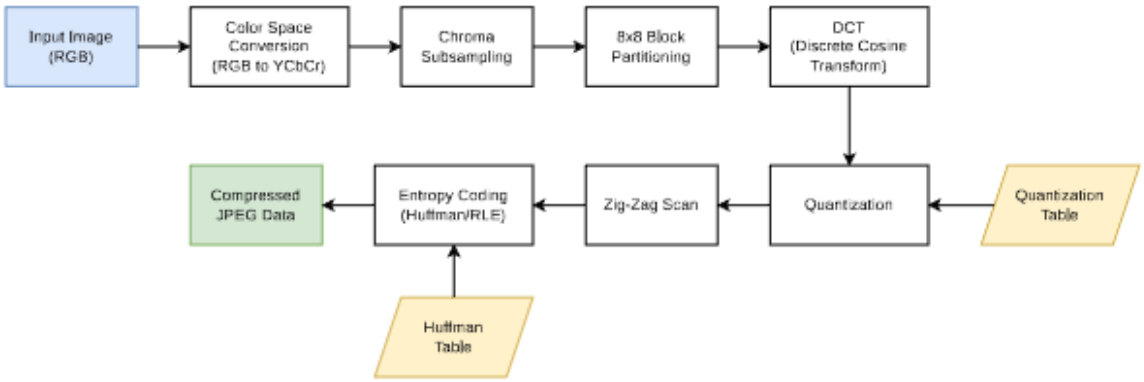
- Adaptive Solutions: based on HS formula, or Greedy algorithm
- Fized Quantization Step: faster and used in JPEG

4.7 JPEG

JPEG is a lossy still-image coding standard based on block DCT, coefficient quantization, and lossless entropy coding. It defines a decodable bitstream syntax, guaranteeing interoperability while allowing different encoder implementations.
If we have 1 component $\rightarrow$ BW images, for color images $\rightarrow$ 3 components: YCbCr.

4.7.1 Encoding Strategy

- Take a block
- Subtrack the average value (128)
- Apply DCT on the blocks
- Quantization using Mid-Tread UQ $\rightarrow \tilde{C}_{ij} = \text{round}\left(\frac{C_{ij}}{q_{ij}}\right)$
- Zig-Zag Scan
- Entropy Coding (Huffman)
- Compressed Data in JPEG format



Full JPEG encoding chain: RGB$\rightarrow$YCbCr conversion, chroma subsampling, 8×8 block partitioning, DCT, quantization (driven by the quantization table), zig-zag scan and entropy coding (Huffman/RLE) into the compressed JPEG stream.

4.7.2 Quantization-step table

The q table is the quantization-step table: steps $\propto$ human sensitivity to the DCT components (64 byte table, $n \times n$ size).
Usually we use the $q^\star$ table, which has been made experimentally on scientific tests, bigger steps for lower sensitivity (BR corner), shorter steps for UL corner components.

4.7.3 Scaling Factor and Quality

JPEG quality Q is an encoder control parameter, not an objective quality measurement. It scales quantization steps: larger Q produces smaller steps, higher rate, and usually higher reconstructed quality; values are not directly comparable across different encoders. The Quantization step table can be scaled by a factor S_F depending on the quality we want:

$$S_F = \begin{cases} \frac{5000}{Q} & 1 \leq Q \leq 50 \\ 200 - 2Q & 50 < Q \leq 99 \\ 1 & Q = 100 \end{cases} \qquad \text{such that actual } q \leftarrow \frac{S_F}{100} q^\star$$

4.7.4 Zig-Zag Coding

Zig-zag scanning is an ordering of the 8×8 DCT coefficients from low toward high spatial frequencies. It tends to place long runs of quantized zeros at the end of the sequence, where run-length coding represents them cheaply. We need a bitstream from the matrix. To encode the sparse matrix we use prediction + Huffman for the DC component, while the others are encoded using 'run-length' coding.
Run-length coding: scanning sequentially the matrix and representing the value as (i, j) where i =number of zeros before, and j the actual value.
In this case the sequential scan is made with a zig-zag technique, final representation is:

for the n-th block: $\quad [DC_n - DC_{n-1}, \quad (i,j) \rightarrow \text{(zig-zag)}, \quad \text{EOB: End of Block}]$

4.8 Entropy Coding of the Coefficients

JPEG entropy coding is the final lossless stage that converts quantized DCT symbols into variable-length bits. It exploits differential DC values, zero runs among AC coefficients, and non-uniform symbol probabilities.

4.8.1 DC Coefficients

The DC coefficient represents the average level of an 8×8 block. Neighboring block averages are correlated, so JPEG encodes the difference from the preceding block's DC coefficient rather than the absolute value. It is a pseudo-Huffman code, we encode the value $DC_p = DC_n - DC_{n-1}$ as a "Category code" + "Amplitude Code", where the categories are defined as:

k-th Category $\rightarrow \lceil \log_2 (|DC_p| + 1) \rceil \qquad \rightarrow \{\pm 2^{k-1} \cdots \pm 2^k - 1\} \subset$ category k, encoded using k bits

4.8.2 AC Coefficients

AC coefficients describe spatial variations around the block average. After zig-zag scanning, JPEG represents each non-zero AC value together with the number of zeros that precede it. The Run-length code uses the same method to encode couples of (r, k) values where $\begin{cases} r \equiv \text{ number of zeros leading} \\ k \equiv \text{ actual value} \end{cases}$.

There are 2 special values: $\begin{cases} (15, 0) \rightarrow \text{ at least 15 zeros} \\ (0, 0) \rightarrow \text{ End Of Block (EOB)} \end{cases}$

4.9 Frame Building

The JPEG frame is the structured coded representation of one image. Headers carry decoding parameters and metadata, while scan and segment payloads carry entropy-coded block data. Image is stored into a **Frame**:

- **Frame Header**: Tells the Image size, components, color sampling scheme, digitization format
- **Frame Payload** contains one scan per component (JPEG Baseline):
  - **Scan Header**: identifies the component and specifies quantization table
  - **Scan Payload** is made up of segments:
    - **Segment Header**: define Huffman tables (4 types: DC, AC, Y, CbCr)
    - **Segment Payload**: 8x8 blocks

4.10 JPEG additional informations

4.10.1 JFIF: JPEG File Interchange Format

Includes some metadata in JPEG, to ensure interoperability, thumbnail, JFIF version, resolution, density unit.

4.10.2 EXIF: Extended Image File Format

- Rich metadata for digital cameras
- Wide range of informations (camera details, datetime, GPS infos, Thumbnail)
- Other Manufacturer-specific data

5 Wavelet-based Image Compression

Wavelet-based image compression represents an image at several spatial resolutions using localized basis functions. It avoids fixed DCT block boundaries and supports progressive transmission by quality or resolution.

5.1 Signal Analysis

5.1.1 Signal Analysis through Projection

Given an orthonormal basis with the φ_k vectors, any signal can be represented as:

$$x(t) = \sum_k c_k \varphi_k(t)$$

We can see the transform as a scalar product:

$$c_k = \langle x(t), \varphi_k(t) \rangle = \int_{-\infty}^{+\infty} x(t) \cdot \varphi_k^\star(t) dt$$

5.1.2 Resolution Tradeoff: Time-Frequency Heisenberg-like Uncertainty principle

Time–frequency resolution describes how precisely an analysis identifies where an event occurs and which frequencies it contains. A narrow time window gives precise localization but coarse frequency discrimination; a long window gives the opposite.

$$(A = \Delta f \cdot \Delta t) \geq \frac{1}{4\pi}$$

The area remains constant, we cannot increase both at the same time.
We can only change shape from (wide flat $\leftrightarrow$ narrow tall).

5.1.3 Frequency Analysis: STFT and Rigid Tilting

The Short-Time Fourier Transform (STFT) applies a Fourier transform inside a sliding fixed-size window. It localizes spectral content in time or space, but every frequency uses the same window resolution. Problem: JPEG uses 8x8 fixed size blocks, having fixed N we cannot change resolution. We need a flexible approach, using Wavelets we have Adaptive Multiresolution, Mother Wavelet generation:

$$\psi_{a,b}(t) = \frac{1}{\sqrt{a}} \psi \left(\frac{t-b}{a} \right)$$

Using the scaling property of the Fourier Transform:

$$\mathcal{F}(x(t)) = X(f) \qquad \Longrightarrow \qquad \mathcal{F}\left\{ x\left(\frac{t}{a}\right) \right\} = |a| X(af)$$

5.2 Discrete Wavelet Transform (DWT) and Multi Resolution Analysis (MRA)

The Discrete Wavelet Transform (DWT) decomposes a sampled signal into low-frequency approximations and high-frequency details through analysis filter banks and downsampling. Multi Resolution Analysis (MRA) repeats this decomposition at several scales.

5.2.1 1D-MRA

Objective: sparsification of the signal. We start from $x[k] = c_{i=0}[k]$ then going on with increasing the i we get the c_i becomes shorter in time, almost by a factor of $\frac{1}{i+1}$. We want a symmetric FIR perfect reconstructing filter, with 4th vanishing moment, such that we are able to work on a grade of 3:

- **Daubechies 9/7** (best fit for image compression)
- **Daubechies 5/3** (integer valued filter taps)

5.2.2 2D-MRA

Two-dimensional MRA applies low-pass and high-pass filtering along image rows and columns. Each level produces one approximation subband LL and three detail subbands LH, HL, and HH, which emphasize different edge orientations. Imagine a black-'squared'-ring, if we divide rows or column into the possible signals, we have only 2 possible signal: all black or a rectangular window. If the signal is constant the output is the same as the input, otherwise if we apply a LPF we get a smoother rect, while if we apply HPF we get some peaks near the color change.
We can now build a square of the combination along H,V of the LPF and HPF: we get LL,HL,LH,HH subbands.
We can also recursively apply this transform to the already transformed square.
By applying it we further sparsify the signal. How to choose decomposition level? It is a tradeoff.

5.2.3 EZW: Embedded Zerotrees of Wavelet Coefficients

Embedded Zerotree Wavelet (EZW) coding is a progressive bitstream method for quantized wavelet coefficients. It exploits the fact that an insignificant coefficient at a coarse scale often has insignificant descendants at finer scales, coding an entire tree with one symbol. It exploits similarities among the subbands, it allows progressive quality while downloading/reading an image.

- Quality Scalability
- Lossy-to-lossless coding
- Small complexity $\rightarrow$ very fast
- At low bitrates image quality is better than JPEG
- Still not used much because money was into JPEG

Idea: each new bit should convey the max possible information, first we encode big samples then progressive information about the subbands exploiting self similarity amond the subbands. Problem: localization overhead.

5.2.4 Recall: Bitplanes

A bitplane is the binary image formed by one bit position of every quantized coefficient. Sending bitplanes from most to least significant progressively refines numerical precision and reconstructed quality. Take the most significant bit $b = \lfloor \log_2(\max(n)) \rfloor$ of the value of the matrix, then we plot the i-th bitplane as a binary matrix made by the values ANDed with 2^{b+1-i}. Example: $b = 4$, first bitplane is made of $(x \ \& \ 2^4)$. Bitplane is equivalent to a uniform quantization. We'll see the top left corner is the one that populates at first.

5.2.5 EZW Algorithm

Take $k = 0, n = \lfloor \log_2 (|c|_{\max}) \rfloor, T_K = 2^k$
Idea: find out a smart way to reach big coefficients without expressing position.

- $\mathcal{L}$ DWT coefficient list accoring to SB scan order
- $\mathcal{S} = \emptyset$ list of significant coefficients
- While (actual rate $<$ available rate)
  - Dominant Pass $\rightarrow$ while $\mathcal{L}$ not empty take c first coeff of $\mathcal{L}$:
    - if $|c| > T_K$ encode symbol as Significant Positive (SP) or Negative (SN) depending on sign(c)
    - if $|c| < T_K$ check descendants, if there is significant descentant $\rightarrow$ ZR else Isolated Zero (IZ)
  - Refining Pass: current bitplane index $b = \log_2 T_k \Rightarrow \forall c \in S$ encode in b bitplane
  - Updating $T_{k+1} \leftarrow \frac{T_k}{2}$
  - Updating $k \leftarrow k + 1$
- Iteration and Termination: k-th dominant pass encodes the b-th bit plane

5.3 JPEG2000

5.3.1 Introduction

JPEG2000 is a wavelet-based still-image coding standard designed for scalability, precise rate control, region-of-interest access, and lossy-to-lossless operation. Its embedded bitstream can be truncated to obtain a chosen bitrate, usually measured in bpp, without re-encoding. It is used in databases of very large images because it allows extraction of different quality and resolution levels. It allows:

- Region-of-interest (ROI) coding
- Lossy-to-lossless coding
- Tiling
- Exact coding rate

How does it work? 2 Tiers:

- 1st tier: DWT + fine quantization to do lossless coding of the codeblocks
- 2nd tier: EBCOT, scalability and ROI management

5.3.2 Quantization

JPEG2000 rate control selects truncation points in independently coded coefficient blocks. Target bitrate is the final number of coded bits divided by image pixels, while distortion is usually measured as MSE; the selected points minimize distortion under that bit budget. DWT coefficient are encoded with very small quantization steps. We encode bitplanes for every codeblock (blocks of the actual image) using lossless coding, since it is lossless we have a lot of bits.
To achieve compression we just truncate bits of the samples. How do we know which bits to truncate? Lagrange:

$$\min \sum R_i \text{ s.t. } \sum_i R_i \leq R_{tot} \qquad \Longrightarrow \qquad \frac{\partial J}{\partial R_i} = \frac{\partial D_i}{\partial R_i} + \lambda = 0 \qquad \Longrightarrow \qquad \frac{\partial D_i}{\partial R_i} = -\lambda$$

In this way the target bitrate is reached with very low error, by truncating the actual computed bits.
We can also use multiple truncation points to reach the best MSE for each bitrate.

5.3.3 Comparison between JPEG and JP2K (JPEG2000)

- For high bitrates: no lots of differences
- For low bitrates: JPEG becomes very bad but still JPEG2000 has a better quality
  - Ringing Artifacts
  - Even comparing 0.2bpp JPEG with 0.1bpp JP2K, this last one is better visually
- JP2K allows the reachability of every code rate we want.

5.3.4 Error Robustness in compressed data

Error robustness is the ability of a coded stream to contain the spatial or temporal effect of corrupted and missing bits. It is commonly evaluated against bit-error or packet-loss probability and usually costs extra rate through markers, resynchronization, redundancy, or independent coding units. Errors can be introduced in TXing information, or saving it into files. Compressed streams are vulnerable, can induce error propagation. We need some 'marker' sequences of bits to recover errors. Tradeoff: robustness-rate.

- JPEG with $P_e = 10^{-4}$: corrupted bit = all the following part of the image corruption
- JPEG2000 with $P_e = 10^{-4}$: corrupted bit = loss of detail in ≥ 1 subbands, visually no great loss.

5.4 Conceptual Maps

DWT

- Signal model
- Filter Banks (analysis/synthesis)
- 2D-DWT: approximation, detail of the subbands
- Energy Concentration (sparsity), intra/inter band correlation

EZW/JP2K

- Wavelet Transform
- Bitplane Coding
- Coding Strategy
  - EZW
  - JP2K
- Comparison with JPEG

6 Learned Image Coding (LIC) / Neural Image Coding (NIC)

Learned image coding uses neural networks to learn analysis transforms, synthesis transforms, and probability models jointly from training images. Unlike a fixed handcrafted transform, its parameters are optimized end to end for a selected rate–distortion objective, often using VAEs[7] or GANs[8].

6.1 Coding Architecture

The analysis transform g_a maps pixels to a latent representation designed for quantization and entropy coding. The synthesis transform g_s reconstructs pixels from quantized latents; together they play roles analogous to transform and inverse transform in a conventional codec.

$$\text{original } x \longrightarrow \boxed{\begin{matrix}\text{Analysis}\\ z = g_a(x)\end{matrix}} \longrightarrow \boxed{\begin{matrix}\text{Quantization}\\ \hat{z} = Q(z)\end{matrix}} \longrightarrow \boxed{\begin{matrix}\text{Synthesis}\\ \hat{x} = g_s(\hat{z})\end{matrix}} \longrightarrow \hat{x} \text{ decoded}$$

Legacy framework (JPEG): g_a is a Block Based DCT, while g_s is a DWT.
Learned Paradigms allow the use of the optimal g_a, g_s by learning directly from data.

[7] Variational Auto-Encoders
[8] Generative Adversarial Networks

6.1.1 Optimization of the Loss Function

The loss function is the scalar objective minimized during training. D penalizes reconstruction error, R estimates expected coded length in bits, and λ sets how much rate is traded for quality. It all goes toward the optimization of the loss function:

$$\mathcal{L} = D(x, \hat{x}) + \lambda R(\hat{z})$$

6.2 NN Recap

For image processing we use CNNs.

6.2.1 Gradient Descent and Backpropagation

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta \mathcal{L}\left(\theta_t\right)$$

6.2.2 GDN: Generalized Divisive Normalization

Generalized Divisive Normalization (GDN) is a learned nonlinear normalization that scales each feature using the energy of neighboring feature channels. It helps reduce statistical dependencies and makes latents easier to entropy-code.

$$w_i = \frac{v_i}{\sqrt{\beta_i + \sum_j \gamma_{ij} v_j}}$$

Given a feature v_i the parameters β_i, γ_{ij} are learned. This allows us to:

- Represent Masking effect
- Linking to the perceptron
- Gaussianization of the data (decorrelation of the signal = use entropy coding)

6.3 JPEG-AI standard

It has nothing to do with the original JPEG. Standardized as ISO/IEC 29170-2.

6.3.1 Core Idea

Going from pixels to 3D-Latent Tensor (sparse signal). JPEG AI uses $\begin{cases} 160 \text{ Luminance channels (human sensitivi} \\ 96 \text{ Chrominance channels} \end{cases}$

6.3.2 Auto-Encoder Framework

- Encoder: Analysis Transform (g_a)
- Bottleneck: Latents y are quantized to $\hat{y}$
- Decoder: Synthesis Transform (g_s)

6.3.3 Rate-Distorsion Variable Auto-Encoder

In a learned codec, rate is the expected number of bits required to entropy-code quantized latents under the learned probability model. It is usually reported as bpp for images; distortion is an expected fidelity loss such as MSE, MS-SSIM loss, or a task-specific metric. Objective: minimizing $\mathcal{L}$ defined as

$$\mathcal{L} = R + \lambda D \qquad \text{where} \begin{cases} \text{Rate } R = D_{KL}\left[q\left(\hat{y}|x\right) || \, p\left(\hat{y}\right)\right] \\ \text{Distorsion } D = E\left[\rho(x, \hat{x})\right] \end{cases}$$

Tradeoff: higher λ = higher quality, lower λ = high compression. Compressing always in latent space.
Problem: Non differentiability for the quantization function, backprop gradient not working properly. We need:

- **Hard** quantization at **test** time
- **Soft** quantization at **training** time → Additive Uniform Noise

Additive uniform noise has the statistical description of the quantization, and makes the backpropagation work.

$$\hat{z} \approx z + u \qquad u \sim \mathcal{U}(-0.5, 0.5)$$

6.3.4 Scale Hyperprior: spatial adaptability

A scale hyperprior is transmitted side information describing local latent statistics, such as coefficient scales. Although the hyperprior consumes bits, it can lower total rate by making the main latent probability model more accurate at each spatial location. IDEA: set NN to learn statistics of the quantized data, additional side information $\hat{z}$ to predict $p_{\hat{y}}$ $\forall$ location.
We change paradigm: since JPEG-AI is a Nonlinear Tranform Coding, we can outperform HEVC and VVC (intra coding) by 50% in coding efficiency. We need a Hierarchical VAE model.

6.3.5 Hierarchical VAE

Multimodality: compression can be adapted do human consumption or machine consumption. JPEG-AI (at 0.5bpp) saves up to 27% compared with VVC. HF are preserved but huge complexity.
Complexity is measured in kilo Multiplication-Accumulation-Count over pixel (kMAC/px). There are 3 profiles:

- Dec0 → 8 kMAC/px (low-end CPUs)
- Dec1 → 23 kMAC/px (also for Mid-range smartphones)
- Dec2 → 214 kMAC/px (high-end GPUs)

6.3.6 Conclusions

- E2E Optimization: $\mathcal{L} = R + \lambda D$
- Data driven transforms (Non linear way)
- Current Challenges:
  - Energy Consumption
  - Cross Platform Determinism
  - Out of distribution vulnerability (Hallucinations and artifacts)
- State of the art: tradeoff
  - Pros → R-D efficiency
  - Cons → COMPLEXITY

7 Motion Estimation

Motion estimation finds a model that relates image content across frames. Its output predicts where pixels or blocks moved and is used to reduce temporal redundancy in video coding.

7.1 Variational Methods

Variational motion estimation defines motion as the field minimizing an energy functional. The objective balances agreement with observed frames against regularity assumptions such as spatial smoothness.

7.1.1 Velocity Vector Field - Optical Flow

Optical flow is a dense apparent-velocity field assigning a horizontal and vertical velocity to each image position. Velocity is measured in pixels per unit time, while displacement over one frame interval is measured in pixels.

$$V : (x, y) \in \mathcal{J} \subset \mathbb{R}^2 \to (u, v)$$

First part of the process is the formulation, then there is the discretization part.

7.1.2 Motion Vector Field

A motion vector field is the discrete displacement assigned to pixels or blocks between reference and current frames. In video coding it is side information, so its accuracy must be balanced against the bits required to transmit it.

$$\forall \text{ point } \implies \exists \text{ vector that represents pixel velocity of motion}$$

Formulation:

$$\arg\min_{(u,v)} \{D\left[f_0, f_1, (u, v)\right] + R(u, v)\} \qquad \text{where} \begin{cases} D \equiv \text{ Data-attachment Term} \\ R \equiv \text{ Regularization Term} \end{cases}$$

7.1.3 Data Attachment and Regularization

- **Data Attachment**: measures coherence of the estimated field compared to f_0, f_1
  Each vector should link pixels of the same color = same pixel.
  Problem: we have much more pixels than colors (assuming 2^8 colors).
- **Regularization** R: We include A-priori knowledge on the MVF.
  It penalizes MVF which are different than our A-priori knowledge, it doesn't depend on data.

7.1.4 Optical Flow Problem

Optical Flow $\propto$ color of the pixels: the goal is to find the best possible predictor. Problem Statement:

- Motion = estimated on the Y component of YCbCr
- we have a Generic Point $\vec{p} = [x, y]^T$
- we have a trajectory $x(t)$

The optical flow is the estimate velocity of trajectory of point $\vec{p}$ at time t.
The associated displacement is the MVF.

7.1.5 Optical Flow - Displacement Field

Consider at a given time $t_0 \to x(t_0) = \vec{p} - D$
Then for a time step $T \to x(t_0 + T) = \vec{p}$
We can define the displacement as this difference:

$$D(\vec{p}, t_0, T) = x(t_0+T) - x(t_0) = \begin{bmatrix} c(\vec{p}, t_0, T) \\ d(\vec{p}, t_0, T) \end{bmatrix} = \begin{bmatrix} c(x,y) \\ d(x,y) \end{bmatrix} \qquad \text{where } \begin{cases} c = \text{ Horizontal Displacement} \\ d = \text{ Vertical Displacement} \end{cases}$$

7.1.6 Optical Flow - Constant Illumination Hypothesis (CIH)

CIH is one of the **Fundamental Hypothesis**.

'Luminance doesn't change along the motion trajectory $x(t)$'

Mathematically we can use the brightness f to define it:

$$f(x, y, t + T) = f\left(x - c(x,y), y - d(x,y), t\right)$$

However this hypothesis is rarely satisfied, due to discretization (sampling, aliasing), noise, degradation of signal.
c, d are unknowns, we can estimate them by linearizing f, we assume f is linear w.r.t. its own data.

7.1.7 Optical Flow - Equation

By using Taylor's expansion:

$$f(\vec{p}, t + T) = f(\vec{p}, t) - \left[c(\vec{p}) \frac{\partial f(\vec{p}, t)}{\partial x} + d(\vec{p}) \frac{\partial f(\vec{p}, t)}{\partial y} \right] + o\left[\|D(\vec{p})\| \right]$$

By dividing all by T and rearranging terms:

$$\frac{f(\vec{p}, t + T) - f(\vec{p}, t)}{T} = -\frac{1}{T} \left[c(\vec{p}) \frac{\partial f(\vec{p}, t)}{\partial x} + d(\vec{p}) \frac{\partial f(\vec{p}, t)}{\partial y} \right] + \frac{1}{T} o\left[\|D(\vec{p})\| \right] = -\frac{\vec{D} \nabla f}{T} + \frac{1}{T} o\left[\|D(\vec{p})\| \right]$$

We can see the left-side term is the incremental ratio (derivative), then from CIH:

$$\frac{\partial f}{\partial t} = -\vec{V} \nabla f \qquad \Longrightarrow \qquad u \frac{\partial f}{\partial x} + v \frac{\partial f}{\partial y} + \frac{\partial f}{\partial t} = \boxed{u f_x + v f_y + f_t = 0}$$

7.1.8 Optical Flow - Solution: Horn & Schunck method

How to find u and v? We use the energy of the function.
Assuming the total variation of the velocity over a region $\mathcal{R}$ is:

$$\iint_{\mathcal{R}} (u f_x + v f_y + f_t)^2 dx dy = \min \text{ s.t. } \iint \left(\|\nabla u\|^2 + \|\nabla v\|^2 \right) dx dy \leq \tau$$

It is a combined minimization problem, it can be solved using lagrangian multipliers:

$$J = \iint_{\mathcal{R}} (u f_x + v f_y + f_t)^2 dx dy + \lambda \left[\iint \left(\|\nabla u\|^2 + \|\nabla v\|^2 \right) dx dy - \tau \right]$$

Solutions $\longrightarrow \begin{cases} \lambda \nabla_2 u = (u f_x + v f_y + f_t) f_x \\ \lambda \nabla_2 v = (u f_x + v f_y + f_t) f_y \end{cases}$ and with $\begin{cases} \nabla u = \bar{u} - u \\ \nabla v = \bar{v} - v \end{cases} \longrightarrow \begin{cases} u = \bar{u} - f_x \frac{\bar{u} f_x + \bar{v} f_y + f_t}{\lambda + \|\nabla f\|^2} \\ v = \bar{v} - f_y \frac{\bar{u} f_x + \bar{v} f_y + f_t}{\lambda + \|\nabla f\|^2} \end{cases}$

Horn and Schunck method provides a smooth motion estimation (smooth and coherent result), very used in practice for object tracking, video compression and motion analysis.

7.2 Block Matching

Block matching estimates motion by searching a reference frame for the block most similar to each current block. It produces one displacement vector per block rather than per pixel, reducing complexity and signaling rate at the cost of a coarser motion model. We split images into blocks $B_{p,q}$ with $(p, q) \in N \times M$
We can estimate our global motion using simple motions of small blocks, simple to program.
The idea is to compare a single block luminance $f_k(B_{p,q})$ with a subblocks in a block window $f_h(B_{p-i,q-j})$

7.2.1 Formulation

Imagine we have two blocks in position $B_{p,q}$ and $B_{p-i,q-j}$, then image indexes h, k and window W (full or subset of the image). We can find the luminance values and compare them using the function d:

$$d\left(f_k(B_{p,q}), f_h(B_{p-i,q-j})\right) = J(i,j) \qquad \Longrightarrow \qquad (\hat{i}, \hat{j}) = \arg \min_{(i,j)\in W} J(i,j)$$

Different techniques differ in:

- minimization criterion of J
- set of candidate vectors W
- Block size and shape

Basically there is a function assigning a score depending on different parameters: we try to minimize that score.

7.2.2 Evaluation of the MVF

How can we tell if the result we get is good enough?

- **Motion-Compensation (MC) prediction associated to MVF** (pretty similar to the other equations)

  Similar to CIH - Brightness $\tilde{f}_k(n,m) = f_h(n + u_{h\to k}(n,m), m + v_{h\to k}(n,m))$

- **Prediction Error**
  $$e(n,n) = f_k(n,m) - \tilde{f}_k(n,m)$$

- **MC-ed MSE**
  $$\mathcal{E} = \frac{1}{NM} \sum_{n,m} e^2(n,m)$$

- **PSNR** $= 10 \log_{10} \frac{255^2}{\mathcal{E}}$

Main Tradeoff on complexity is to change block size $P \times Q$.
Larger blocks reduce coding costs, complexity, and increase MSE

7.2.3 Block Matching criteria

The matching criterion is a distortion score used to rank candidate reference blocks. Lower SAD or SSD means a closer pixel match; a regularized criterion also charges the estimated number of bits needed to code the motion vector.

- **SSD** (Sum of Squared Differences), norm-based criteria

$$J_{SSD}(i,j) = \sum_{(n,m)\in B_{p,q}} [f(n,m,k) - f(n-i,m-j,h)]^2$$

  - PROS: Good for maximizing PSNR
  - CONS: Difficult to compute, increase in entropy of MVF, doesn't account global illumination changes.

- **SAD** (Sum of Absolute Differences)

$$J_{SAD}(i,j) = \sum_{(n,m)\in B_{p,q}} |f(n,m,k) - f(n-i,m-j,h)|$$

  - PROS: globally costs less bits, quality still really good
  - CONS: problems with parts with same colors (unknown behavior due to noise)

- **REG** Regularized norm-based criteria $\rightarrow$ solves most of the problems

$$J_{REG}(i,j) = \left\| \vec{f_k}(B_{p,q}) - \vec{f_h}(B_{p-i,q-j}) \right\|_p^p + \lambda R(i,j)$$

7.2.4 Full-Search research strategy

Full search evaluates every integer displacement inside the search window and selects the candidate with minimum matching cost. It gives the optimum for that window and criterion, but its operation count grows with both window area and block size. We consider windows of the image, with size $= (2A + 1) \times (2B + 1)$.
Then we build a $(2A + 1) \times (2B + 1)$ SSD matrix, this brings $\propto n^2$ complexity. By searching the minimum of the SSD matrix, it is the best possible vector that can be used to approximate the window.

7.2.5 Fast Research Methods: 3SS (Three step search, 2D-log)

The use of Full-search is computationally expensive, we need strategies to test less positions in the window. Assuming error function is unimodal $\Rightarrow \exists$ one global minimum.
3SS method allows to test only 9 positions ($x \in \{0, \pm A/2\}$,$y \in \{0, \pm B/2\}$).Avoiding to compute 89% operations.
We recursively apply this method on the sub-windows centered in the minimum valued element of the matrix, and also reducing the size of the sub-matrix by a half, until we get a 1-pixel subwindow.

7.2.6 Fast Research Methods: Diamond Search

We rotate the initial 9-points structure by 45 degrees, thus we use a diamond pattern to search, new iteration can cost from 3 (diagonal displacement) to 5 (horizontal/vertical displacements) positions to compute.
Recursively refining until final refinement is done through a cross-shaped 5-position small diamond. Avoiding to compute 90% operations.

7.2.7 Fast Research Methods: Hex Search

As the diamond search, here we use an hexagon-like shape with 7 positions to compute. Horizontal, vertical and diagonal displacements cost 3 positions to compute. Final refinement is done through the same cross-like shape of the diamond search. Avoiding to compute 92% operations.

7.2.8 Fast Research Methods: TZSearch

Robust to local minima, phases:

- Search predictors: search time/space neighbors, if low error stop
- Adaptive loop: if no predictor is good, use diamond/square search increasing steps

7.2.9 BM Improvement: Sub-pixel precision

Sub-pixel motion estimation allows vectors with fractional-pixel components by interpolating reference samples. It can reduce prediction error, but adds interpolation complexity and may require more motion-vector bits. With half pixel precision, we can reduce cost function because we have more options.
Problem: we have to increase resolution, we can use some interpolation methods, steps:

1. select a certain pixel $(\hat{i}, \hat{j})$
2. we test $(\hat{i} \pm \frac{1}{2}, \hat{j} \pm \frac{1}{2})$
3. recursively we test all subpixels with $1/4, 1/8$...

7.2.10 BM Improvement: Variable Blocksize

Variable block-size motion estimation uses large blocks in uniform-motion regions and smaller blocks near boundaries or complex motion. Splitting improves prediction but increases partition and motion-vector signaling rate. Foreach block b of size B in image F:

- calculate $J = D + \lambda R$
- divide b into 4 subblocks
- compute $J_i \ \forall i$ subblocks, $J_{sub} = \sum J_i$
- if $J_{sub} < J(v)$ apply algorithm to subblocks, else keep and store MV

7.3 Parametric Methods

Parametric motion models describe an entire region using a small set of parameters rather than independent vectors. They can represent coherent camera or object motion with low signaling cost when the chosen model fits the scene. We want MVF as a function of each pixel, degrees of freedom = parameters of the function.

7.3.1 Affine Model

An affine motion model represents translation together with rotation, scaling, and shear using six parameters in two dimensions. Each pixel displacement is computed from its coordinates and the same regional parameter vector.

$$\vec{v}(p) = \vec{b} + Bp = \begin{bmatrix} b_1 \\ b_2 \end{bmatrix} + \begin{bmatrix} b_3 & b_4 \\ b_5 & b_6 \end{bmatrix} p$$

Special case if $B = 0 \rightarrow$ translation (Block Matching), other possible intermediate cases are:

- Zoom In or Zoom Out
- Rotations
- Displacement (Translations)

7.3.2 Best possible parameters

Which are the best possible parameters? We know the affine model has 6 parameters $\pi = [b_1 \cdots b_6]$
Indirect Estimation

$$\pi^\star = \arg\min_{\pi} \sum_{n,m \in R} [u(n,m) - u_\pi(n,m)]^2 + [v(n,m) - v_\pi(n,m)]^2$$

Direct Estimation, first approach

$$\pi^\star = \arg\min_{\pi} \sum_{n,m \in R} [u_\pi(n,m) f_x(n,m) + v_\pi(n,m) f_y(n,m) + f_t(n,m)]^2$$

Direct Estimation, second approach (SAD/SSD minimization)

$$\pi^\star = \arg\min_{\pi} \sum_{n,m \in R} [f(n - u_\pi(n,m), m - v_\pi(n,m), t-1) - f(n,m,t)]^2$$

7.4 Deep-Learning for Motion Estimation

- Specialized architectures (CNNs)
- Large-scale datasets for training (Synthetic or Real, used by some game video quality enhancement tools)
- High computational power of inference by using GPUs (in the last few years)

7.4.1 Time evolution

- First Generation: **FlowNet (2015)**, first CNN for motion estimation (input images, output optical flow)
- Second Generation: **FlowNet 2.0 (2017)**, better tradeoff complexity-performance
- Third Generation: **PWC-Net (2018)**, faster and quite efficient in complexity, easier to implement
- Nowadays:**RAFT (2020)**, slower than PWC-Net, precise on thin structures, iterative updates and GRU[9].

7.4.2 Usage Comparison

DL methods has the best efficiency in analysis tasks (Optical Flow).
DL methods used in real-time video coding need to be tuned by practical constraints.
Advantages: Robustness, high precision
Challenges: Generalization on different datasets, computational cost to be optimized.

[9]Gated Recurrent Unit

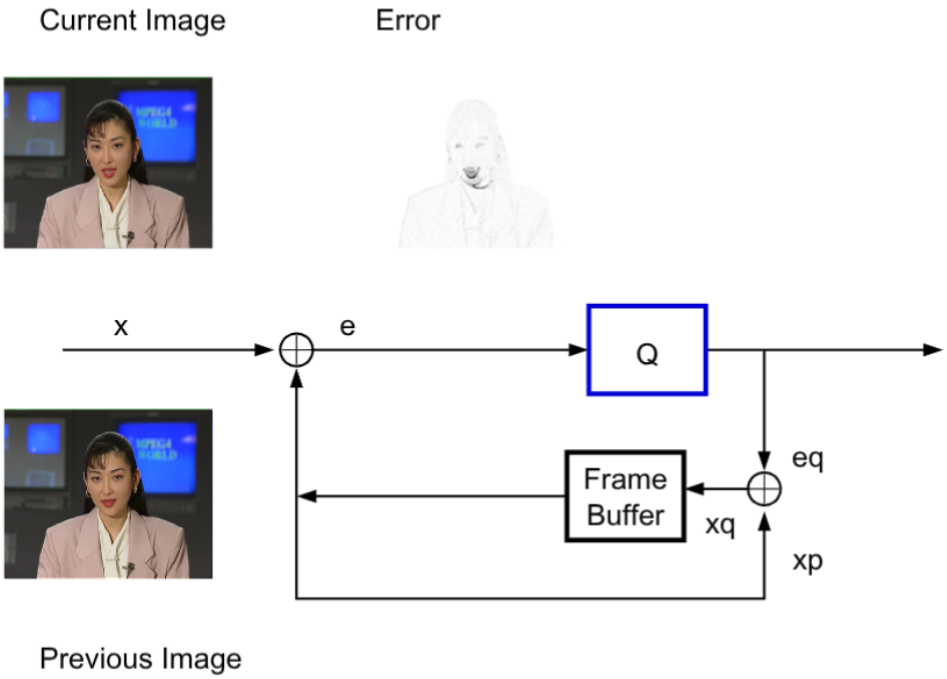
8 Video-coding Principles

Video coding reduces both spatial redundancy within frames and temporal redundancy between frames. Video bitrate is the number of coded bits produced per second, usually measured in kbps or Mbps; at fixed frame rate it depends on average bits per frame. We want to exploit both spatial redundancy and temporal redundancy.

8.1 Block Schema

$$x \longrightarrow \boxed{\text{Temporal Compression}} \longrightarrow \boxed{\text{Spatial Compression}} \longrightarrow \boxed{\text{Buffer}} \longrightarrow$$

Can we swap Temporal and Spatial compression blocks? Yes but it isn't used because it doesn't make sense.



Hybrid coding intuition: only the temporal prediction error $e = x - x_p$ (previous frame) is quantized and transmitted; the reconstructed frame x_q is stored in the frame buffer to predict the next one (closed loop, same anti-drift principle as DPCM).

8.2 Block-Matching motion estimation

Block-matching motion estimation searches reference frames for predictors of current blocks. Its cost $J(v)$ combines prediction distortion with motion-vector rate, so the visually closest vector is not always the cheapest vector to code.

$$J(v) = d(B_k^{(p)}, B_h^{(p+v)}) + \lambda_{ME} R(v)$$

How can we build the prediction using motion estimation? Motion Compensation

8.2.1 Motion Compensation

Motion compensation constructs a predicted block by copying or interpolating samples from a reference frame at the selected motion-vector position. Only the residual between current and predicted blocks then needs to be coded.

$$\hat{I}_k(p) = I_h(p + v^\star(p))$$

Sometimes MC works better and sometimes it works worse, solution: adaptive blocks.

8.2.2 Adaptive block coding

- Intra Coding: encode block as it is
- Inter Coding: use other images to predict it

$\forall B_k^{(p)}$ block of image k:

- compute $J(v)$
- find the $v^\star = \arg\min_p v(p)$
- encode block:
  - If intra: encode in a JPEG-like way
  - If Inter:
    - Decode $v^\star$ motion vector
    - Decode predictor error blocks $E_p^{(b)}$
    - Correct blocks

8.2.3 Design Parameters

Block size (smaller better), Cost function (SAD + regularization),
Motion Model (translational), Search Strategy (fast methods: hex, TZ)

8.3 GOP: Group of Pictures

A Group of Pictures (GOP) is a coded sequence organized around independently coded and predictively coded frames. GOP length and frame pattern determine compression efficiency, random-access interval, structural delay, and error-propagation length:

$$\boxed{I}\boxed{B}\boxed{B}\boxed{P}\boxed{B}\boxed{B}\boxed{P}\boxed{B}\boxed{B} \rightarrow n\text{-th GOP}$$

8.3.1 'I' frames: Intra Frames

Access points of our videos, decoding starts here. Useful to stop error propagation. Fast to encode, no ME. All blocks of an I frame are Intra coded. Used for: **random access** (independently decodable), **fast forward** (decode only I frames), **error robustness** (stop error propagation). Low complexity but low compression.

8.3.2 'B' frames: 'between' frames

Uses both past and future frame exploitation for prediction. This leads a change in encoding order (1,4,2,3,7,5,6,10,8,9), leading also to a **structural latency**. In Real time applications we do not use B frames due to this latency. For each block of a B frame the encoder can choose: Intra coding, **forward** prediction (from the past), **backward** prediction (from the future), or **bidirectional** prediction (average of past and future predictors). Very high complexity (double ME) but the highest compression ratio.

8.3.3 'P' frames: Predictive Frames

Predicted from the **previous anchor frame (I or P)**. Decision taken at block level: each block can be Inter or Intra coded. High complexity (ME/MC) but much higher compression than I frames at the same quality.

8.3.4 Rate and quality of the frame types

I frames are $\approx 3 \to 5\times$ larger than P frames and $\approx 10 \to 20\times$ larger than B frames. The quality of I frames is typically forced high, because they predict (directly or indirectly) the whole GOP.
The GOP structure controls the tradeoff among: compression efficiency, delay, random access, error propagation.

8.4 Hybrid Video Encoder

8.4.1 Frame coding modes

Encoder tries all of these to encode a frame:

- Intra: available for all frames
- Inter: ME/MC-based temporal prediction (only NON-intra frames)
- Direct (skip): trying to infer MV from neighbors (gain in rate, loss in distorsion)
- Lossless: all frames can be encoded in this way

8.4.2 Block Size: Block Partition Problem

Rate–distortion optimization selects coding modes and partitions by minimizing $J = D + \lambda R$. D is reconstruction distortion, R is coded size in bits, and λ converts additional bits into an equivalent distortion penalty.

$$D = \sum_{k=1}^{K} D_k(i_k, Q) \qquad R = \sum_{k=1}^{K} R_k(i_k, Q)$$

We want the lowest distorsion for a given bitrate:

$$J(\vec{i}, Q, \lambda) = D(\vec{i}, Q) + \lambda R(\vec{i}, Q) \qquad \text{where } \vec{i} = \{i_k\}_{k=1}^{K} \text{ s.t. } i_k^\star = \arg\min_{i_k} J(i_k, Q, \lambda) = \arg\min_{i_k} J_k(\vec{i})$$

How to find Q and λ? Q is an input (set by rate control); for each Q there is an optimal λ, determined **empirically** per codec:

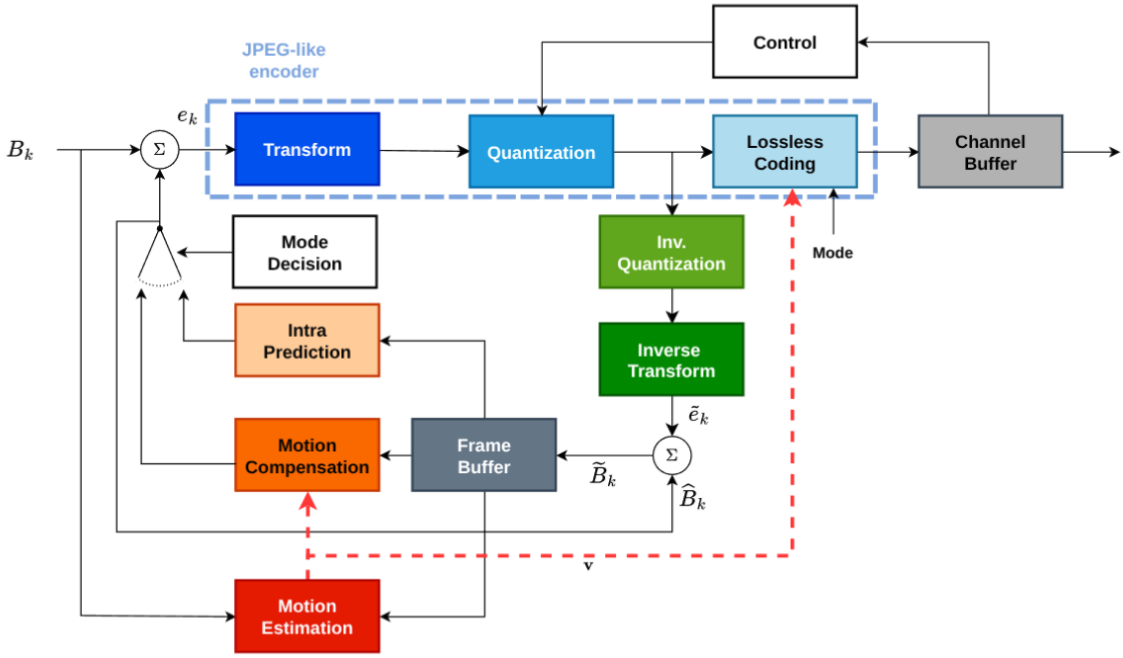
$$\text{MPEG-2: } \lambda = aQ^2 + b \qquad \text{H.264: } \lambda = c \cdot 2^{dQ+e} \qquad \lambda_{ME} = \sqrt{\lambda}$$

Joint minimization over all $\vec{i}$ is too complex $\Rightarrow$ suboptimal **block-wise** minimization of $J_k(i_k, Q, \lambda) = D_k + \lambda R_k$, one block at a time. Block partition cast as a mode too: split if $J_{split} = \sum_i J_{subblock_i} < J_B$, applied recursively from the largest block size.

8.4.3 Coding Mode i_k selection

On a R-D plane we plot all the points related to the available modes, then we select the nearest to our J slope. $D = -\lambda R + J$ is a family of lines with slope $-\lambda$: high λ (steep) $\to$ low bit budget, picks low-rate modes (e.g. Direct); low λ (flat) $\to$ prioritizes fidelity (e.g. Inter8 bidirectional). The chosen mode is the first point touched by the line moving from the origin towards the convex hull of the available modes.

8.4.4 Encoder scheme



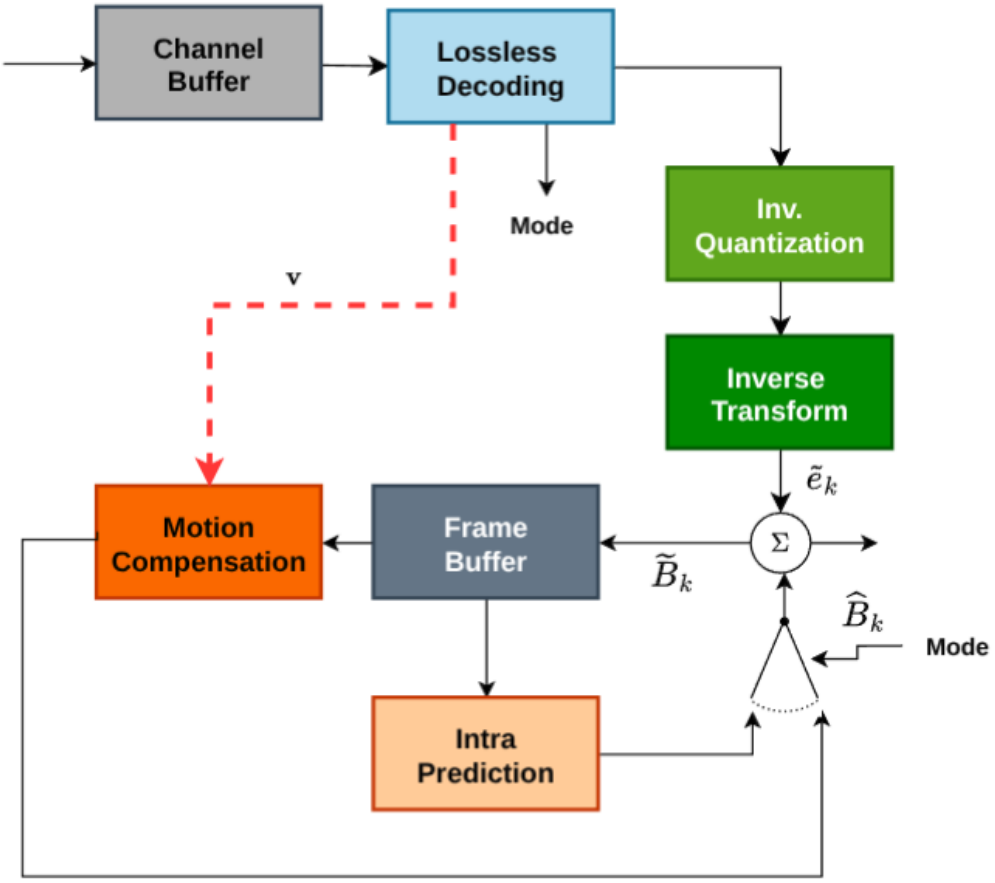
Full hybrid encoder block diagram. The dashed JPEG-like core (Transform → Quantization → Lossless Coding) compresses the residual $e_k = B_k - \tilde{B}_k$. The reconstruction loop (Inv. Quantization → Inverse Transform → $+\tilde{B}_k$) feeds the Frame Buffer used by Motion Compensation, Motion Estimation and Intra Prediction; Mode Decision picks the predictor and Control drives rate via the Channel Buffer. Comment on the blocks:

- Each image is split into blocks; for each block the encoder computes a prediction v: **null** (direct DCT coding), **temporal** (ME/MC, simple or bi-prediction), or **spatial** (Intra, from already encoded blocks of the same frame).
- The prediction error $y = x_k - v$ is JPEG-like compressed: Transform + Quantization.
- **Decoder loop inside the encoder**: Q^{-1}, T^{-1}, then $\tilde{x}_k = \hat{y} + v$ is stored in the **DFB** (Decoded Frame Buffer), because future temporal/spatial predictions must use the same reconstructed data the decoder will have (no drift, same principle as closed-loop DPCM).
- Side information (mode decision, block partition, motion vectors) + quantized coefficients are entropy-coded with a VLC (Huffman/Arithmetic/Exp-Golomb; MVs coded predictively, exploiting spatial coherence of the MVF).
- **Rate control**: encoded stream enters the channel buffer at rate R_C (content-dependent) and leaves at target rate R_T. Closed-loop controller: if occupancy $> \gamma_{high}$ increase quantization step (R_C drops); if $< \gamma_{low}$ decrease it (R_C grows).

Rate control is the encoder mechanism that adjusts quantization and coding choices to meet a target bitrate or file size while avoiding buffer overflow and underflow. Its target is measured in bit/s for streams or total bits for stored content. Why do we have standards? Interoperability, we only standardize the decoder (able to decode anything), to leave encoding competition.

8.5 Hybrid Video Decoder

8.5.1 Decoder scheme



Hybrid decoder: only the reconstruction half of the encoder. After Lossless Decoding, Inv. Quantization and Inverse Transform give $\tilde{e}_k$, summed with the prediction (Motion Compensation from the Frame Buffer, or Intra Prediction, selected by Mode) to output $\tilde{B}_k$. No mode decision / motion estimation $\Rightarrow$ far cheaper than the encoder.

- First decodes the **side information**: partition, coding mode, motion vector(s).
- Then inverse quantization, inverse transform, and sum with the prediction (MC or Intra) to get the final block, stored back in the DFB.
- The decoder does **not** perform: mode decision, block partition decision, motion estimation (the heavy operations) $\Rightarrow$ decoder much faster than encoder. These are read from the bitstream.
- Since only the decoder is standardized, ME strategy / mode decision / partitioning are free: competition on encoder efficiency (hex-search, fast mode decision, ...).

8.5.2 Key elements

- Spatial Compression: Transforms
- Temporal Compression: GOP and ME
- LossLess coding: Huffman and Arithmetic codings
- Coding Mode estimation: Lagrangian ($D + \lambda R$)

8.6 Video Encoding Standards

- MPEG (1,2,4): among the first standards, since 1988
- H.264/AVC: free standard
- H.265/HEVC: some patenting
- H.266/VVC: not really free, we have some patenting
- AV1, VP8, VP9: Royalty free standards (AOM[10])

8.6.1 MPEG-1

MPEG-1 Part 2 (MP3 Audio Format) was the first digital audio standard used for CDs.
MPEG-1 allows:

- Rate $\leq$ 1.86Mbps
- [720 $\times$ 576]@30fps max resolution
- Audio: 128 $\rightarrow$ 320 kbps

[10] Alliance for Open Media

9 Modern Video-Compression Standards

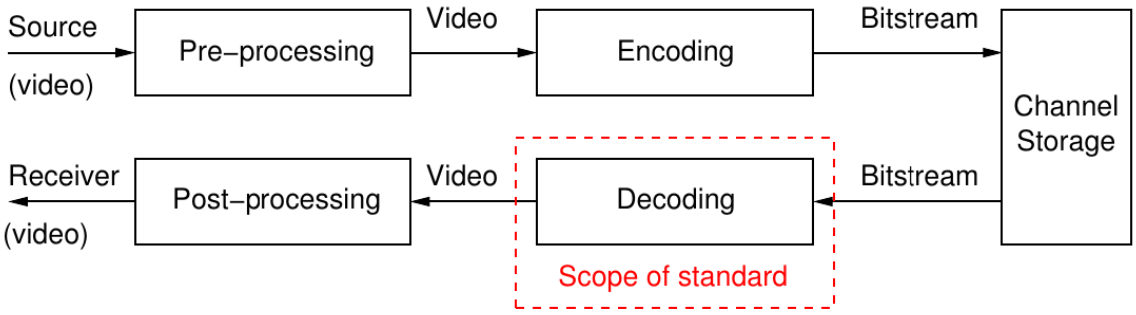
A video codec is an encoder–decoder pair that maps frames to a compressed bitstream and reconstructs frames from that stream. A coding standard defines bitstream syntax and normative decoding behavior so conforming implementations interoperate.

9.1 Universal Hybrid Video Encoder

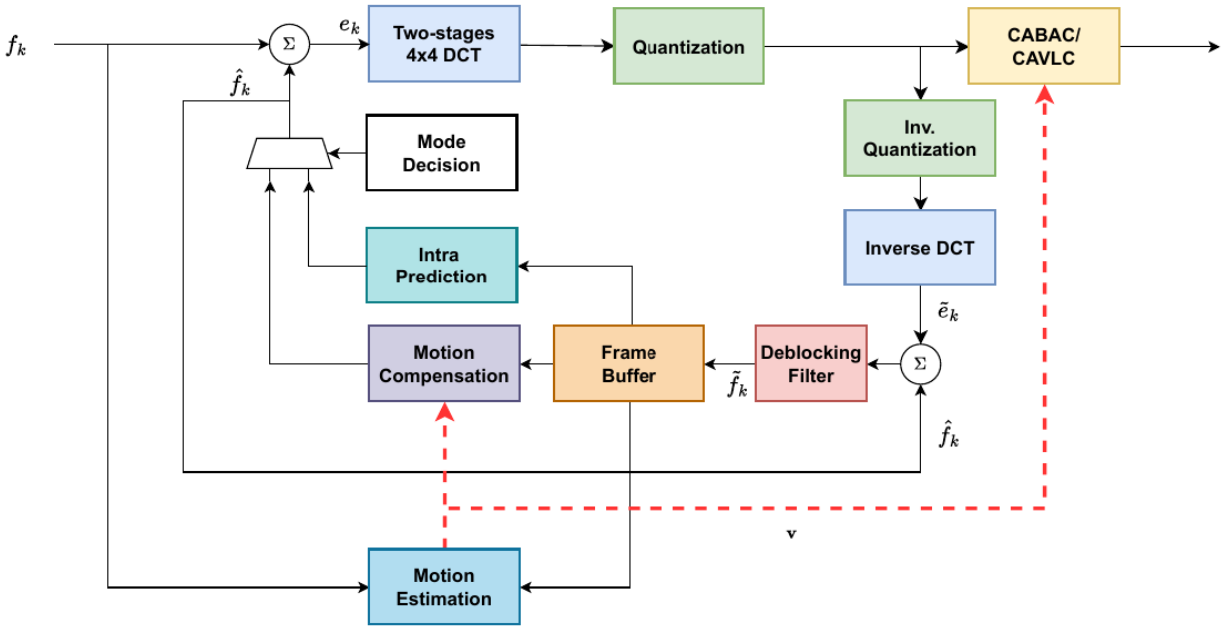
From MPEG-1 going on they use all the same structure, core points:

- Spatial (intra) and temporal (inter) prediction
- Residual calculation
- Transform and Quantization
- Internal closed loops to provide reliability

Rule of thumb: every generation reduces video size by 1/2 and increases complexity by 10×



End-to-end video chain. Only the **decoding** is normative (*scope of the standard*): pre/post-processing and the encoder are left to implementation competition.



Modern hybrid encoder (H.264-style): integer two-stage 4×4 DCT, in-loop **Deblocking Filter** inside the reconstruction loop before the Frame Buffer, and **CABAC/CAVLC** entropy coding. Same closed-loop structure as the universal encoder, refined block by block.

9.1.1 Codecs applications

- Real Time → H.264 (very diffused, also in HW implementations since 2003)
- Broadcast → H.264 + HEVC, growing HEVC market share
- Web Streaming → H.264 + HEVC + AV1 (still royalty free)
- Next-Generations → need to use massive bitrates of raw video → VVC, AV1

9.2 Rate-Distorsion Optimization

Rate–distortion optimization (RDO) compares valid coding choices using a combined cost. Rate R is the number of output bits, distortion D measures reconstruction error, and λ controls the relative price assigned to rate.

$$J = D + \lambda R$$

Using the R-D optimization method we have a huge number of options, how do we choose the best mode?
The solution is to work in blocks → Block Partitioning Problem

9.2.1 Block Partitioning Problem: H.264

In H.264 we use macroblocks 16×16, it allows further splitting until 4×4 blocks, useful to compute edges.
Still this size is too low for High-Resoluton videos, we need something more.

9.2.2 Block Partitioning Problem: H.265/HEVC

In HEVC, a Coding Tree Unit (CTU) is the largest processing region, a Coding Unit (CU) carries coding decisions, and a Prediction Unit (PU) defines a prediction partition. Their hierarchy adapts block shape and size to local image structure. HEVC uses a Quad-Tree paradigm, it is like a 2D binary tree.
A certain CTU (Code Tree Unit) can be split into 4 CU (Coding Units), then a CU becomes a CTU and we can apply this recursively. Standards should provide only the syntax, we need to do the actual implementation.
We continue splitting until another split provides no more improvement, and complexity becomes reasonable.
CU can be split even in asymmetrical quantities into the PU (Prediction Units), then we use S and T prediction.

9.2.3 Block Partitioning Problem: H.266/VVC

It uses the same paradigm but here we have a 10-way superblock partitioning, we can binary and ternary split.
How can we build the decision tree? Depth First Search:

- Encode Block, freeze it
- Go to the next neighbor block
- Iterate until all blocks are encoded

If the block is flat, doesn't make any sense to partition it.
In this case we also have a huge number of possible partitions, but we give up in looking all possible choices.
This is like a classification among pixel groups, ML and NN are good in solving these problems.
Researchers are looking for a light NN to exploit this.

9.3 Spatial and Temporal Frame Prediction

9.3.1 Intra Prediction

Intra prediction estimates a block from already reconstructed neighboring samples in the same frame. It removes spatial redundancy without referring to other frames and therefore supports random access and limits temporal error propagation. We want to exploit spatial prediction among neighbors. Instead of trying just one prediction we use different predictors in parallel for the same block, then we pick the best one. We have to signal this to the Encoder.
We take one director-per-block instead of one-per-pixel (it is too much bits wasted):

- **H.264/AVC**: 9 modes for 4×4 blocks (**8 directional + DC**); 4 modes for 16×16 blocks (2 bits). A 16×16 MB split in 16 4×4 subblocks has ideally 9^{16} combinations: done sequentially to cut them down.
- **H.265/HEVC**: **35 modes = 33 directional + DC + Planar**, to handle complex textures.
- **H.266/VVC**: **65 directional modes + wide-angle** modes for non-square blocks.

Predictors grow exponentially with the signaling bits: $P \propto 2^B$. With 67 modes, ≈ 7 bits/block of pure signaling would destroy the gain on 4×4 blocks.
MPM: Most Probable Mode idea, ENC predicts which is the most probable direction using the spatial context: a candidate list is built from the modes chosen by top and left neighbors (HEVC: 3 candidates, VVC: 6). If the chosen mode is in the list, 1–2 bits suffice; only if it fails long codewords are used.

9.3.2 Inter Prediction

Inter prediction estimates a block from one or more previously reconstructed reference frames using motion information. It removes temporal redundancy but requires motion-vector and reference-index side information. Here we use Fractional Motion Compensation, each frame is computed using multiple reference frames.
For each image in the GOP, the bitstream identifies the list of frames available for prediction.
Here complexity $\propto$ size of bits. How do we encode the motion vectors?
A better strategy is to produce a list of predictors and encode the indexes.

9.3.3 Merge Mode

Used in AV1/VVC, not only picks the pixels but also applies affine transformation on the blocks, thus producing a complexity increase.

9.4 Filtering, Transforms and Quantization

9.4.1 Residual Coding

The residual is the sample-wise difference between the original block and its selected prediction. Residual coding transforms, quantizes, and entropy-codes this difference; better prediction produces a lower-energy residual and usually a lower rate. DCT problem: we want floating number operations (too complex for an encoder, must be quick).
Solution: just use integer power of 2, such that multiplications are bit shifts and we have also $\pm$ operations.
We don't provide quantization table, but a similar Quantization Parameter ($\propto$ dB step size)
QP behavior (H.264/HEVC): the quantization step **doubles every +6 of QP**; empirically +1 QP $\approx -12.5\%$ rate. QP is the main knob for rate control.
The Quantization Parameter (QP) is a coded index controlling quantization-step size. Higher QP means coarser quantization, lower bitrate, and greater distortion; QP itself is dimensionless and is not a direct quality score. Entropy coding standardized around **CABAC** (Context-Adaptive

Binary Arithmetic Coding): binarizes syntax elements and adapts probabilities on spatial context; $5 \rightarrow 15\%$ rate saving over older VLC (CAVLC).

9.4.2 In-loop Deblocking Filter

Why artifacts: transform and quantization are applied *independently on disjoint blocks* $\Rightarrow$ at low bitrate strong discontinuities (**blocking artifacts**) appear at block boundaries.
Why in-loop (and not just post-processing): uncorrected artifacts would be stored in the Decoded Picture Buffer and copied around by future motion compensations, propagating the damage. The filter is **normative** (inside the reconstruction loop, identical at encoder and decoder to keep sync).
Key insight: artifact locations are known a priori — the block edges — so we apply adaptive smart smoothing there, without blurring real image edges.

- **H.264/AVC deblocking**: analyzes edges between 4×4 blocks; filtering strength adapts dynamically to coding mode, motion vectors and quantization step of the neighboring blocks.
- **H.265/HEVC**: deblocking only on 8×8 grid (less complexity) + **SAO** (Sample Adaptive Offset): classifies pixels by edges/bands and adds offsets to fix ringing artifacts.
- **H.266/VVC**: adds **ALF** (Adaptive Loop Filter) and LMCS (HDR detail preservation).
- **AV1**: CDEF directional filters, Wiener loop-restoration filters, film grain synthesis (grain re-synthesized at decoder at zero rate cost).

9.4.3 Slices

A slice is an independently entropy-decodable sequence of coded blocks within a frame. Slice boundaries support resynchronization and parallel processing, but reduce prediction efficiency and add header overhead. Error corruption behavior: artifacts highlight due to error propagation in compression.
How can we prevent this? Slices are subparts of the frame, that are independently encoded, such that we can parallel-encode them, because we don't use them for prediction.
At Intra Frames CABAC[11] is reset.

9.5 Network parallelism

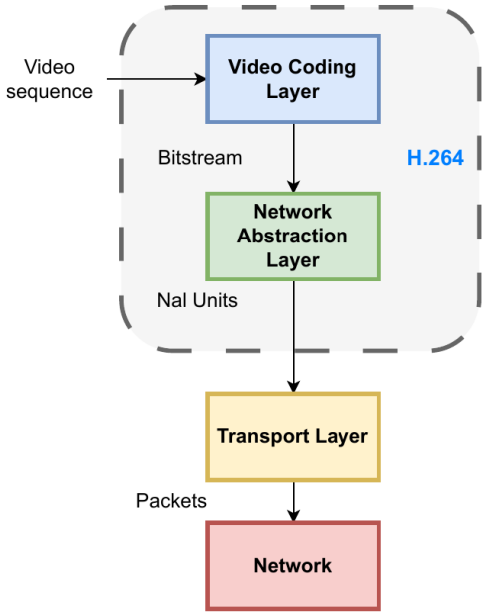
All the video data is encoded into a bitstream.

9.5.1 NALU

A Network Abstraction Layer Unit (NALU) is a self-delimited packet of video-coding data or metadata. The NAL layer separates codec syntax from transport packaging and lets networks identify parameter sets, slices, and access points without fully decoding video.

- VCL NALUs: contain all the compression data for a slice, can be IDR[12] (I + CABAC reset) or non-IDR
- non-VCL NALUs: contain SPS, PPS, SEI
  - SPS: Sequence Parameter Set (picture size, color depth,...)
  - PPS: Picture Parameter Set (coding specific parameters)
  - SEI: Supplemenetal and Enhancement Infos (Subtitles, HDR infos, ...)

[11] Context-Adaptive Binary Arithmetic Coding
[12] Istantaneous Decoding Refresh, video access point



H.264 layered architecture: the **Video Coding Layer** produces the compressed bitstream, the **Network Abstraction Layer** wraps it into NAL units, which the Transport Layer packetizes for the network. NALUs are then packetized into a sequential bit stream, there are 2 ways to signal the NALU (H.264/HEVC):

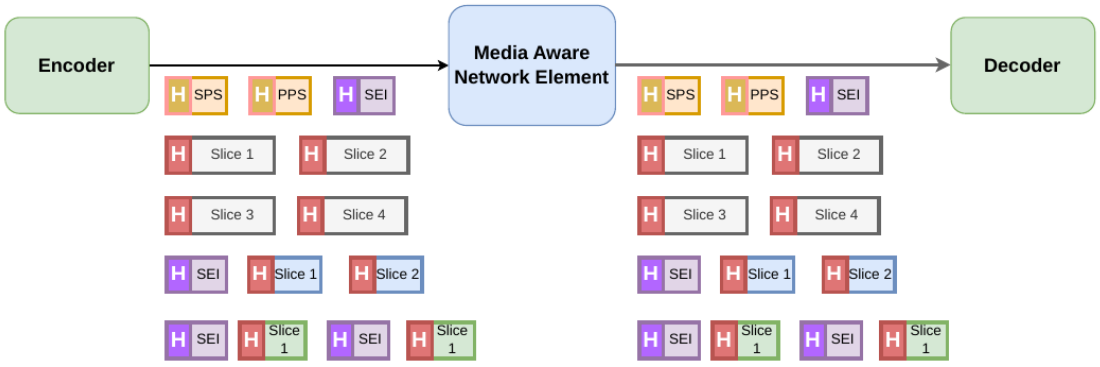
- Start Code $0x00000001$: it is used but we have problems if this sequence is repeated in a nalu (start code mismatch), we have to use Emulation Prevention Techniques
- Length Header: before the NALU we specify the actual NALU length, such that we can read it or skip it.

Actual Sequence using Annex-B ordering is:

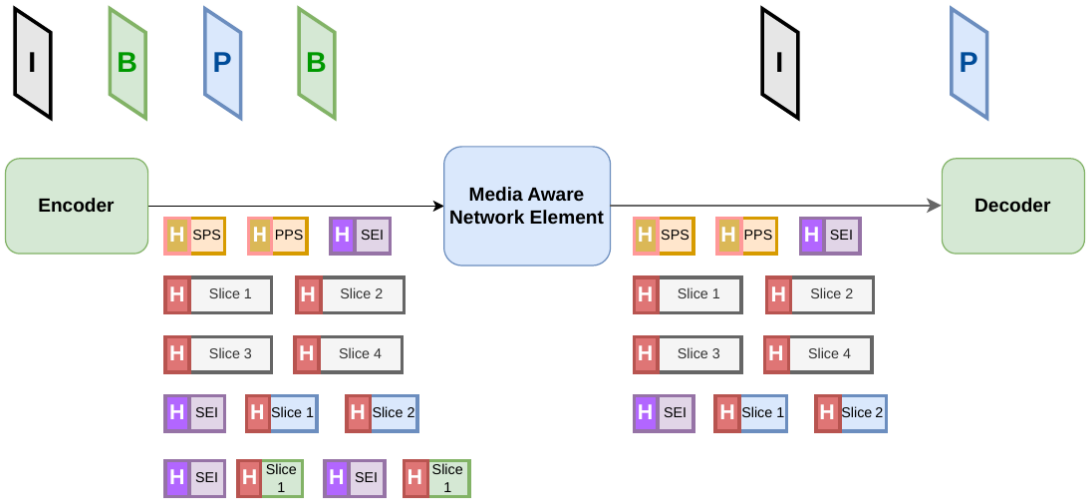
SPS	PPS	AUD	SEI	I slice $1 \rightarrow 4$
		AUD	SEI	P slice $1 \rightarrow 2$
		AUD	SEI	B_1I slice 1
		AUD	SEI	B_2 slice 1

AV1 drops this approach and uses OBUs (Open Bitstream Units), in CDNs there are MANE[13] that can allow to change the bitstream on the fly to add redundance or remove unnecessary NALUs.

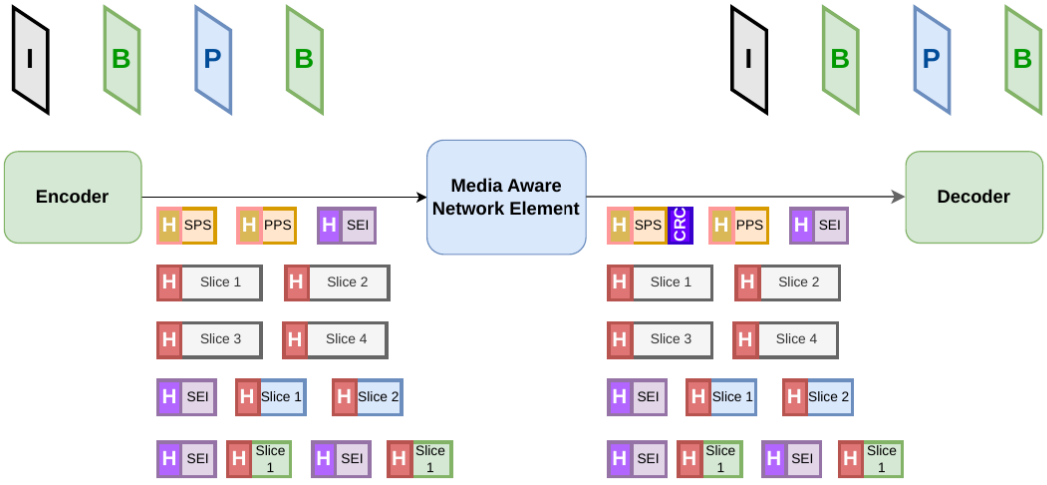
[13]Media Aware Network Elements



NAL-unit stream Encoder → MANE → Decoder: parameter sets (SPS, PPS), SEI and slice NALUs are forwarded packet by packet.



Pruning: the MANE drops the non-decodable B-frame NALUs, forwarding only the I and P frames (rate adaptation without re-encoding).



Adding redundancy: the MANE can inject extra protection (e.g. a CRC NALU) into the stream for more robust delivery. Decoder accesses video at RAPs (Random Access Points), there are 2 main access techniques: **A Random Access Point (RAP)** is a bitstream position from which decoding and display can begin without decoding the entire preceding stream.

- IDR: Technique to clear previous buffer at a certain intra frame. It breaks bidirectional temp. prediction
- CRA (Clear Random Access): we just skip some images because they are not decodable.

10 Audio and Speech Coding

Audio coding represents sampled sound with fewer bits while preserving a required perceptual quality, intelligibility, or latency. Audio bitrate is measured in bit/s and depends on sampling rate, channel count, and average coded bits per sample. Different uses have different requirements:

- VR and AR need low latency
- Dynamic Range, deal with 100 dB differences
- Transparency of the encoded audio w.r.t. the original signal

10.1 Audio Modeling

10.1.1 Requirements

- Speech is stationary but for small intervals ($\sim$ 20ms), it uses **source based** coding, linear filters to guarantee intelligibility and low latency.
- General Audio is more homogeneous (fast transients, tonal segments), it uses **sink**[14] **based** coding, needs to guarantee fidelity and transparency.

10.1.2 Vowels

Pitch is the perceived fundamental frequency of a periodic sound and is measured in hertz. Its reciprocal is the pitch period T_0, measured in seconds or samples. Vowels are very correlated with zero-centered frequency.
Usually there are harmonics, ordinary FT doesn't work, here we use Short-Time Fourier Transform (STFT).
Pitch is the frequency of the most important harmonic, then the others are computed based on this.

10.1.3 Music

Histogram is less sparse, correlation is still strong. There is no harmonic structure.
We treat these sounds as images so we use an image-like analysis, using perceptual models.

10.1.4 Human Speech Production System

3 Entities:

- Power: we use the lungs
- Source: we have our vocal folds
- Filter: the vocal tract act as a filter

There are 2 main types of sounds:

- Voiced sounds: depend on a pitch period T_0, which depends on the person (M/F, old/young)
- Unvoiced sounds: it is like filtering differently white noise (fricatives etc...)

10.2 Linear Predictive Coding

Linear Predictive Coding (LPC) models each short speech frame as an excitation passed through an all-pole vocal-tract filter. The encoder transmits filter parameters, gain, and excitation information instead of coding every waveform sample directly.

$$\hat{x}(n) = -\sum_{i=1}^{P} a_i x(n-i) \qquad \Longrightarrow \qquad \text{s.t. } y(n) = x(n) - \hat{x}(n) = \sum_{i=0}^{P} a_i x(n-i) \text{ with } a_0 = 0$$

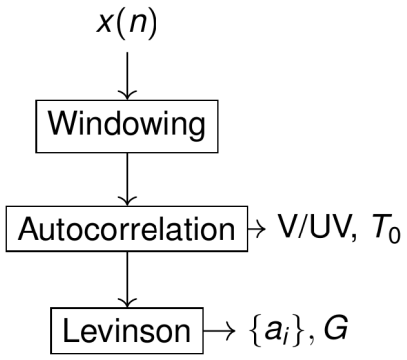
[14] Sink = human ear

10.2.1 Yule-Walker Equation

$$\mathbf{R}_x \vec{a} = -\vec{r}_x$$

How do we find the a_is? We use this YW equation. The $\mathbf{R}_x$ is the Toeplitz matrix of autocorrelation, the $\vec{a} = [a_1 \cdots a_p]^T$ is the unknown coefficient vector, and the $-\vec{r}_x$ is the autocorrelation vector. There is also an efficient algorithm which has small complexity: **Levinson-Durbin** Algorithm.

$$x(n) \longrightarrow \boxed{\text{Windowing } \sim 20ms} \longrightarrow \overset{\text{Voiced/Unvoiced},T_0}{\boxed{\text{Autocorrelation}}} \longrightarrow \boxed{\text{Levinson-Durbin}} \longrightarrow \{a_i\}, G$$



LPC analysis chain: windowing ($\sim$ 20ms frames), autocorrelation (which also yields the voiced/unvoiced decision and pitch T_0), then Levinson(-Durbin) to solve Yule-Walker for the filter coefficients $\{a_i\}$ and the gain G. How do we estimate autocorrelation? Check linear predictors (2.4.5), very similar behavior.
How do we detect the pitch? Compare $k > P$, how do we check Voiced/Unvoiced? Compare to $\mathbf{R}_x(0)$

10.2.2 Residuals

The LPC residual is the prediction error left after removing the estimated short-term vocal-tract structure. It approximates the excitation source: periodic impulses for voiced speech and noise-like samples for unvoiced speech.

$$Y(z) = A(z)X(z) \qquad \Longrightarrow \qquad X(z) = A^{-1}(z)Y(z)$$

Residuals are:

- If unvoiced: zero mean white noise
- If voiced: periodic spikes

If Y are impulses, then A^{-1} (inverse filter) is the $H \Longrightarrow X$ is also the H.
We know a_i are numbers, how do we quantize them? We have 2 problems:

- a_i can vary a lot, we don't have an a-priori range knowledge
- Filter is not BIBO[15] stable

[15] Bounded-Input, Bounded-Output

10.2.3 LPC Synthesis

We know how to get the a_is, but we also want BIBO stability

$$\begin{cases} P(z) = A(z) + z^{-(P+1)}A(z^{-1}) \\ Q(z) = A(z) - z^{-(P+1)}A(z^{-1}) \end{cases} \qquad \text{such that} \qquad P(z) + Q(z) = 2A(z)$$

It must be easy to recover A from P, Q. All the roots are on the unit circle, how do we get stability?

$$\text{(BIBO) Stable} \qquad \Longleftrightarrow \qquad \text{roots of } P(z) \text{ are alternate with the } Q(z) \text{ ones}$$

From the Itakura paper (1975): $0 < \omega_1^{(P)} < \omega_1^{(Q)} < \omega_2^{(P)} < \cdots < \pi$
If the roots are close to one another we have sharp peaks in the filter response.

10.3 Vector Quantization

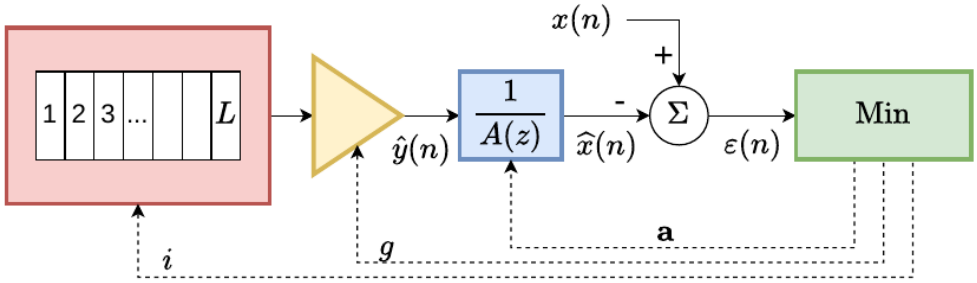
Vector quantization maps an input vector to the closest representative vector in a finite codebook. Only the selected codebook index is transmitted, so its fixed rate is $\log_2 L$ bits per vector for a codebook of L entries, or $\log_2 L/P$ bits per scalar component for vectors of dimension P. We have a set of $\omega = [\omega_1 \cdots \omega_p]$, then we have a codebook (shared dictionary of most important ω_i).
Between Voiced and Unvoiced saomples we have 28 bits to allocate freely:

- Voiced: 21 bits for LPC coefficients, 7 bits for T_0
- Unvoiced: all 28 bits go to the error proctection

10.3.1 CELP Paradigm (Code-Excited Linear Prediction)

Code-Excited Linear Prediction (CELP) is an analysis-by-synthesis speech codec. The encoder tests candidate excitation vectors through the synthesis model and transmits the codebook index and gains that minimize perceptually weighted reconstruction error. 3 main ideas:

- Vector Excitation: choose residual vector from codebook
- Analysis-by-synthesis: encoder is like a mini-decoder
- Closed loop shape search, and then tx the optimal shape.

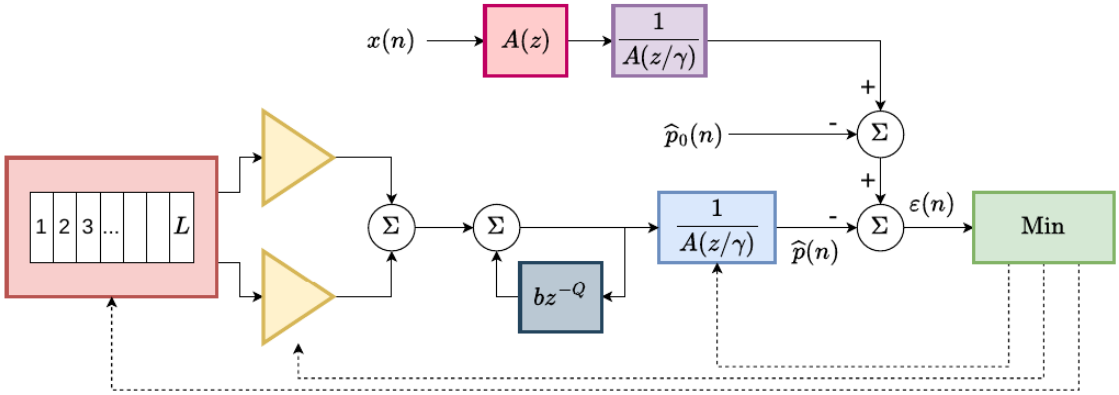


Basic CELP analysis-by-synthesis: each candidate codebook entry i (scaled by gain g) is passed through the synthesis filter $1/A(z)$; the index/gain minimizing the error $\varepsilon(n)$ against $x(n)$ is transmitted (closed-loop search). CELP minimizes the perceptual weighted error, the optimal shape of the error should be something like:

$$W(z) = \frac{A(z)}{A(z/\gamma)} \qquad \text{with } 0 < \gamma < 1$$

Then CELP does also pole shifting to move poles toward the origin.
The adaptive part models pitch based on previous $y(n)$:

$$\begin{cases} \text{Sample Lag } Q \\ \text{Gain } b = g_P \end{cases} \qquad \Longrightarrow \qquad y_{\text{adapt}}(n) = y(n - Q)$$

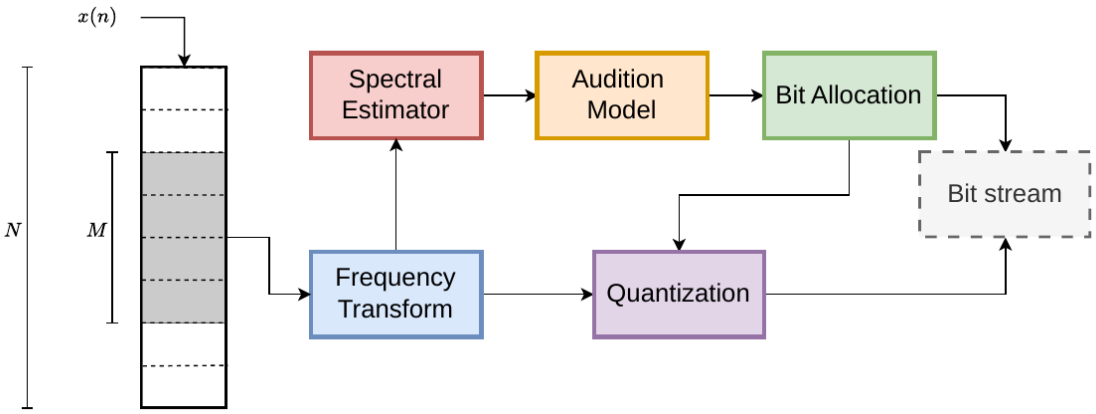


Full CELP: the error is shaped by the perceptual weighting filter $W(z) = A(z)/A(z/\gamma)$, and a **long-term predictor** (adaptive codebook, lag Q, gain b, branch $b\,z^{-Q}$) models the pitch periodicity on top of the fixed (stochastic) codebook. Examples: G.729, G.722.2 (AMR-WB), EVS

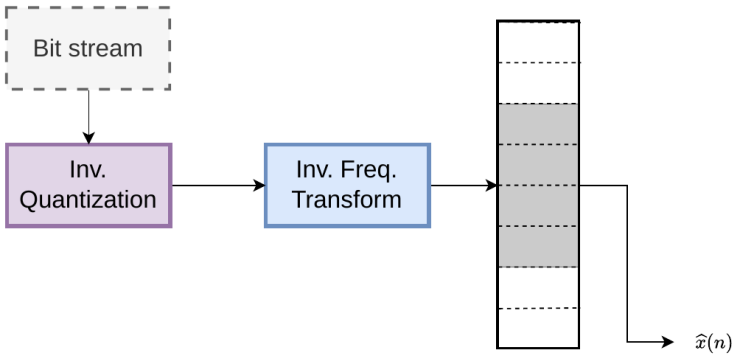
10.3.2 Perceptual Coding for Audio

Perceptual audio coding removes or coarsely quantizes spectral components hidden by auditory masking. A psychoacoustic model estimates a frequency-dependent masking threshold, and bit allocation keeps quantization noise below that threshold where possible. Coclea in human ear acts as a sum of critical bands (BPF), narrow bands for low frequency, wider bands for high frequency. Masking Effects:

- Frequency masking effect is still effective, the quantization mask based on frequency is defined as $\varphi(f)$
- Temporal Masking: we use pre/post masking techniques



Perceptual audio **encoder**: the signal is windowed and frequency-transformed; in parallel a Spectral Estimator feeds the Audition (psychoacoustic) Model, which drives Bit Allocation, so quantization spends bits only where the ear is sensitive (masking).



Perceptual audio **decoder**: inverse quantization then inverse frequency transform reconstruct $\hat{x}(n)$ (the psychoacoustic model is encoder-only). Standards: MP3 (first actually used digital standard for audio), aac (basically it is an mp3 improvement).

10.4 Modern Techniques

10.4.1 OPUS

Makes use of CELT techniques for speech processing (xiph.org based), and uses SILK techniques for Music (Skype derived). It also provide Low-Latency and it is the de-facto standard for Discord,WebRTC,...

10.4.2 Neural Codecs

Google and Meta proposed their own solutions, they can reach very low bitrates (3/4 kbps) but at which cost? Complexity! They will be used as a fallback solution.
Currently we also have to take into account the 'Hallucination'-challenge of NN-based codecs.

10.4.3 Spatial Audio

3 Main methods:

- Channel Based Distribution (5.1, 7.1)
- Mono + Position information approach (Object based)
- Scene-based approach

10.4.4 Quality Assessment

- Mean Opinion Score, human based, it costs much (POLQA, VISQOL)
- MUSHRA (ITU-R BS.1534)
- Transparency Test

Why SNR is not an effective parameter? We are insensitive to phase noise, perceptual methods needed.

11 Quality Assessment and QoE for Multimedia Services

11.1 Quality Assessment

Quality assessment is the process of assigning a numerical score or category to a multimedia stimulus. It estimates technical fidelity or perceived quality using either human judgments or an objective algorithm.
The assessment process can be objective (using a math. formula) or subjective (ask to people).
We have a Quality/Rate tradeoff, since these process are time/space-complex, energy and latency consuming.
Quality of Experience (QoE) is the user's overall satisfaction with a service. It is multidimensional because perceived media quality, startup delay, stalls, quality changes, device, context, and expectations all contribute.

11.2 Image Quality: Subjective Assessment

Subjective assessment measures quality directly from ratings provided by human observers under controlled conditions. It is treated as ground truth for perception, but results have statistical uncertainty and experiments are costly. It is not easy to get consistant results, need to define carefully the equipment, such that the environmental conditions are equal for all the people that is asked. Process is costly, offline and time consuming.
Because of this it is used as the golden standard, but in a 2^{nd} phase of testing (cannot afford at first trial).

11.2.1 Testing Conditions

People must be in the same conditions to get good assessments, we have to take into account people artifacts (expert people vs non-expert people, people who prefer movements vs detail, colorblind people, myopia...)

- Space Information: lots of details and structures → Nature Documentaries
- Temporal Information: lots of movement → football matches (high difference between 2 near frames)

Users should be informed about the assessment methods, it shouldn't be too long, people can be tired.
Human Observers do not always agree. Techniques can be compared using a flowchart.

11.2.2 1^{st} Technique: Single Stimulus

We have ACR (Absolute Category Rating) techniques. What if we have to compute a 'normal' scale? ACR-HR (Hidden Reference) technique.

11.2.3 2^{nd} Technique: Double Stimulus

DSIS (Double Stimulus Impairment Scale) technique, very high accuracy, used for codec validation. It alternates reference video with the test video.

11.2.4 3^{rd} Technique: Pairwise Comparison

PC technique assures a fine perceptual ranking, by comparing some times 2 different images/videos and asking which is the best among those possible answers.

11.3 Human Assessment Measures

Nowadays we can use also crowdfunded campaigns to call volunteers.

11.3.1 MOS: Mean Opinion Score

Mean Opinion Score (MOS) is the arithmetic mean of ratings assigned by N observers. It is dimensionless and uses the rating scale chosen by the test, commonly 1–5 or 0–100; a higher value usually means better perceived quality.

$$\text{MOS} = \frac{1}{N}\sum_{i=1}^{N} x_i$$

This is evaluable with std. measures

11.3.2 Standard Measures

Standard deviation measures disagreement among individual ratings. **Standard error** measures uncertainty in the estimated mean and decreases as observer count grows. A **confidence interval** gives a range expected to contain the unknown population mean at a stated confidence level.

- Standard Deviation: $s = \sqrt{\frac{1}{N}\sum_{i-1}^{N}(x_i - m)^2}$
- Standard Error: $\text{SE} = \frac{s}{\sqrt{N}} = \frac{1}{N}\sqrt{\sum_{i-1}^{N}(x_i - m)^2}$
- Confidence Interval: $\text{CI} = [-1.96 \cdot \text{SE}, 1.96 \cdot \text{SE}]$

11.4 Image Quality: Objective Assessment

Objective quality assessment computes a score from signal data without asking observers during evaluation. A useful metric should correlate with subjective judgments while remaining repeatable and inexpensive. It is difficult to evaluate the human perceptual system using maths.
It is always a chain of:

$$\boxed{\text{Input Signal Reference}} \longrightarrow \boxed{\text{Signal Processing}} \longrightarrow \boxed{\text{Output Signal}}$$

$$\text{Input Signal} : I(n,m,c,t) \qquad \text{Output Signal} : \hat{I}(n,m,c,t) \qquad \text{where} \begin{cases} n, m \rightarrow \text{ position} \\ c \rightarrow \text{ component} \\ t \rightarrow \text{ time} \end{cases}$$

11.4.1 Full Reference Techniques

Full-reference metrics compare the complete original signal with the complete degraded signal. They require access to both versions and are therefore mainly used during codec development, laboratory testing, and offline benchmarking. From the previous chain we take the input I and the output $\hat{I}$ and we compare them in the $\boxed{FR}$ block.

11.4.2 Y component measures

$$\text{MSE}_Y(t) = \frac{1}{NM}\sum_{n,m}\left[I(n,m,1,t) - \hat{I}(n,m,1,t)\right]^2$$

$$\text{PSNR}_{Y,\text{dB}}(t) \approx 48\text{dB} - 10\log_{10}\text{MSE}_Y(t)$$

11.4.3 CbCr component measures

Analog to the Y component we can write:

$$\text{MSE}_{Cb}(t) = \frac{1}{\frac{N}{2}\cdot\frac{M}{2}}\sum_{n,m}\left[I(n,m,2,t) - \hat{I}(n,m,2,t)\right]^2 \qquad \text{MSE}_{Cr}(t) = \frac{1}{\frac{N}{2}\cdot\frac{M}{2}}\sum_{n,m}\left[I(n,m,3,t) - \hat{I}(n,m,3,t)\right]^2$$

11.4.4 Total PSNR measure

$$\mathrm{PSNR}_{\mathrm{YCbCr}}(t) = \frac{3}{4}\mathrm{PSNR}_{\mathrm{Y}}(t) + \frac{1}{4}\left[\frac{1}{2}\mathrm{PSNR}_{\mathrm{Cb}}(t) + \frac{1}{2}\mathrm{PSNR}_{\mathrm{Cr}}(t)\right]$$

11.4.5 Bjontegaard Deltas

Bjontegaard Delta summarizes the average separation between two rate–distortion curves over a common operating range. BD-Rate reports average bitrate difference at equal quality, while BD-PSNR reports average quality difference at equal rate. Given two different codecs:

- Delta PSNR: Average PSNR difference (in dB) between the two codecs
- Delta Rate: Average rate difference (in percentage) at the same quality

We don't actually need to compute the integrals, we just use log-based parametric functions given 4 points.

11.4.6 Perceptual Metrics

Why PSNR is not enough? It is a Per-pixel distorsion measure (no idea of the temporal perception), for videos we have to use perceptual metrics:

- SSIM: Structure-SIMilarity (compares YCbCr and structural information instead of PSNR-like methods)
- VMAF: Video Multi-method Assessment Fusion (from Netflix and Google(?))
  - Multiple features to estimate video quality
  - It exploits ML techniques to get objective testing
- AI-based metrics: CNN based, trained using image subjective datasets (get very good estimations)

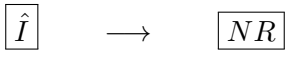
11.4.7 Reduced Reference

Reduced-reference assessment compares compact features extracted from the original with the degraded signal. It needs less side information than full-reference assessment but more than no-reference assessment. Practical for Large Scale systems. The input I and output $\hat{I}$ goes in a parallel way into:

We focus on a limited subset of features, which are preserved in the final image.

11.4.8 No Reference

No-reference assessment estimates quality using only the degraded signal. It is suitable when the original is unavailable, but it relies on learned or handcrafted assumptions about natural content and common distortions. It only used the $\hat{I}$ output:



Techniques: BRISQUE, NIQE, PIQE. Used to assess photos.
Mobile smartphone cameras are assessed this way because the metrics estimate characteristics associated with natural, visually acceptable images.

12 Adaptive Streaming

Adaptive streaming divides media into independently downloadable segments available at several encoded bitrates. The client changes representation over time to match network conditions and buffer state while maximizing QoE. While streaming a content we need to take into account rate-quality (tradeoff). Video is the most difficult problem because we have to reduce uncompressed video (from the order of Gbps of rate, down to some Mbps).
Possible applications of multimedia streaming:

- Bulk Transfer (need data integrity)
- VOD (delay tolerant)
- Live streaming (scalability and low delay)

It is possible to control the encoders such that we are able to compress video with the same rate.

12.1 Video Content Distribution

12.1.1 Challenges

Each user has different links to the central server, and each user has different constraints (display, battery,...)
A possible solution is the Conversational Paradigm (1 source to N destinations):

$$\boxed{\text{Speaker A}} \longrightarrow \boxed{\text{Conference SRV}} \longrightarrow \begin{cases} \boxed{\text{Listener B (TV)}} \\ \boxed{\text{Listener C (Phone)}} \\ \boxed{\text{Listener } n} \end{cases}$$

How to choose the coding rate of my video?

12.1.2 Coding Rate Bottleneck

Coding rate R_c is the average number of compressed media bits produced per second. Link capacity C_k is the maximum sustainable transport rate for user k; stable delivery requires coding rate, protocol overhead included where appropriate, to stay below available throughput. We need:

$$R_c \leq C_k \qquad \text{(capacity limit of user } k\text{)}$$

We need to keep the rate lower than the max capacity, to avoid buffering.
if $R_c = \min_k c_k$ we fix a low resolution also for high-capacity linked users.
Solution: use MANE, also deployed in CDNs and 5G Networks.

12.1.3 Paradigms

Push

- Goal: Minimize latency
- Stack: WebRTC/RTCP/RTP/UDP
- Server pushes, client listens (Real Time approach)

Pull

- Goal: Maximize QoE
- Stack: [DASH,HLS]/HTTP3/QUIC/UDP/(CDNs)
- Client asks for the content, server serves

UDP is quick and fast, RTP just adds additional infos like:

- Packet Numbering
- if Out-of-order packets → sort out

RTCP also adds also a metrics exchange as control messages to let the server adapt the R_c.
There are also timestamps added to packets to handle sync and control jitter.

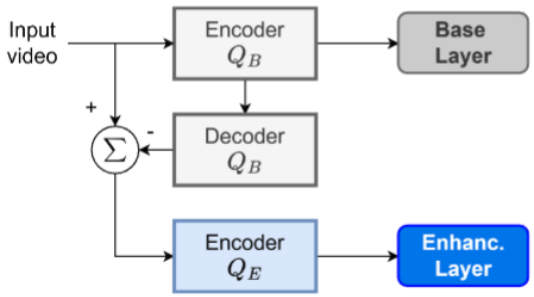
12.2 Scalable Video Coding (SVC)

Scalable Video Coding produces a layered bitstream containing a mandatory base layer and optional enhancement layers. Removing enhancement layers lowers rate, quality, resolution, or frame rate without re-encoding the source. Useful for a $1 \to N$ realtime video streaming. IDEA: just perform 1 encoding and then be able to extract several versions of the content. (JP2K like capabilities $\to$ progressive video coding)

12.2.1 Scalability Types

- **Time** Scalability: exploit temporal prediction
- **Quality** Scalability: use progressive coding, Base-Layer (BL) as prediction of ELs (Enhancement-Layers)
- **Space** Scalability: exploit Multiresolution Transforms, low-res base-layer + enhancement layers

Enhanced Layer is computed as $= I_{\text{nput}} - \text{DEC}(\text{ENC}(I_{\text{nput}}))$



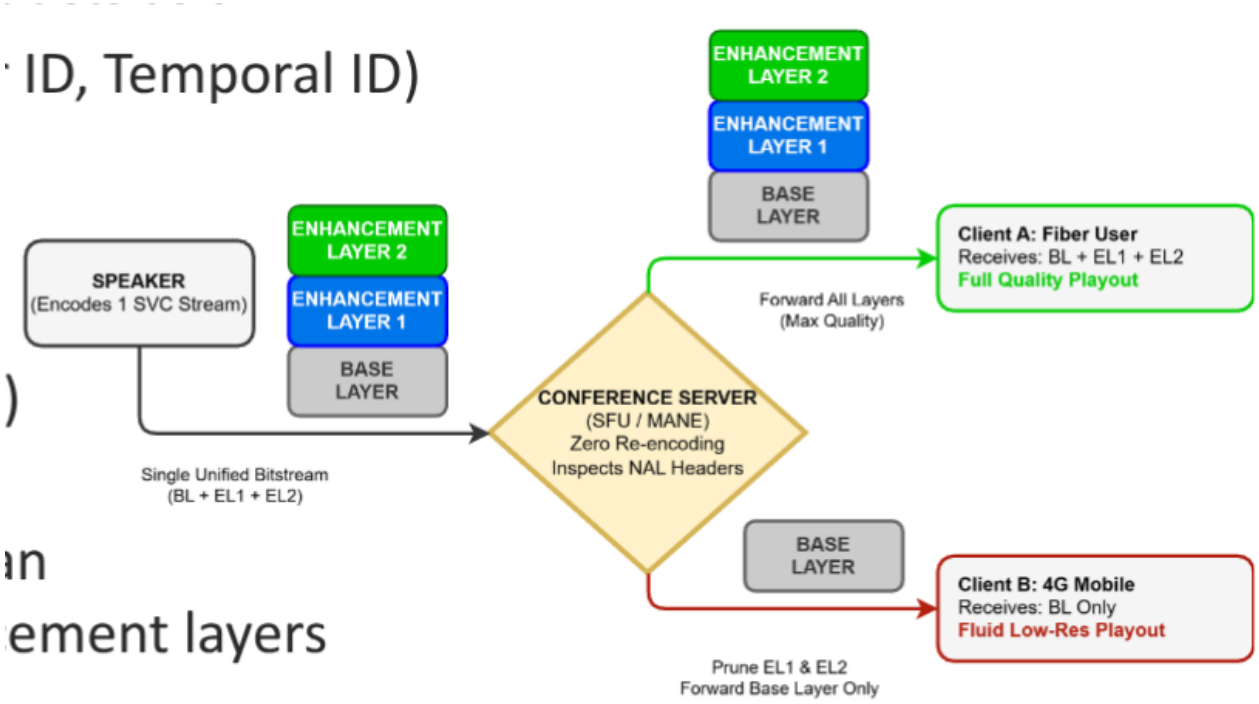
SVC layered encoder: a coarse encoder Q_B produces the **Base Layer**; its local decoder reconstruction is subtracted from the input and the residual is coded by Q_E into the **Enhancement Layer** (quality scalability).

12.2.2 Smart Layer Pruning

Assume we have the following architecture:

$$\boxed{\text{Speaker}} \longrightarrow \boxed{\text{Confluence Server}} \longrightarrow \boxed{n\ U_i}$$

The speaker produces the Base-layer + 2 more enhancement layers, total of 3 Layers.
Confluence server decides how to send the layers to users, it's the **Selective Forward Unit**, usually a MANE.



SFU layer pruning: the speaker sends one unified bitstream (BL + EL1 + EL2). The conference server forwards *all* layers to the high-capacity client (full quality) and only the Base Layer to the constrained mobile client — zero re-encoding, just NAL-header inspection.

12.2.3 Streaming via Legacy HTTP

Why don't we use the same solution for streaming? SFU needs memory proportional to the connected users, to keep the state of each. And the BL + EL solution has some overhead w.r.t. a single high res block of data. Solution: use HTTP because it is stateless and also largely implemented.
We can slice the video and have caching using the **CDNs**[16] at the network edges.
The only drawback of Legacy HTTP is TCP, we increase latency and stop playout if a segment is lost (Head-of-line blocking). This implies ReTX and large delays. Solution: QUIC.

12.2.4 QUIC and HTTP/3

HTTP/3 is based on UDP:

- removes Head-of-line blocking
- it has short setup time (no handshakes)
- has connection migration, not bound to (ip,port) couple, just a connection ID

12.3 Adaptive Bitrate Streaming

Adaptive Bitrate (ABR) streaming lets the client select one encoded representation for each segment. Representation bitrate is measured in bit/s and indicates encoded media size per playback second, not instantaneous download speed.

[16] Content Delivery Networks

12.3.1 Media Presentation Description (MPD) file

The Media Presentation Description (MPD) is a DASH manifest describing content timing, tracks, available representations, segment locations, codecs, resolutions, and bitrates. It contains metadata and URLs rather than media samples.

- **Period**: It is representing a certain content
- **AdaptationSet**: It represents a certain track (video, audio,...)
- **Representation**: It represents the different qualities (different codecs, resolutions, bitrates,...)
- **Segments**: They are the actual sliced media parts, by joining them together we get the actual content.

Further notes:

- Segments (.m4s) are Indepently decodable, they only need an init signature part (.m4i)
- To encode a segment you have to wait the IDR block in a certain GOP.
  Also in the decoding we'll start from a fresh new IDR block.
- If we know networks changes quickly: better short segments, otherwise viceversa.
- For Live streaming we have to use small chunks of about $200 - 500$ ms, this is achieved with CMAF[17]
  this leads to a glass-to-glass latency of about $1 \to 3$ seconds.

12.3.2 Network Metrics: Throughput

Throughput is the rate at which useful data are successfully delivered over a time interval. It is measured in bit/s and is usually lower and more variable than physical link rate because of protocol overhead, congestion, competing traffic, and losses. Here we use continuous models to describe discrete things.

$$S(t) = \frac{d}{dt} D(t) \leq R_{\text{PHY}}$$

This inequality takes into account also for Overhead + Congestions + Other users on the link. Since S can be very sparky we want to use the average:

$$\bar{S} = \frac{1}{T} \int_0^T S(t)dt = \frac{D(T)}{T}$$

12.3.3 Network Metrics: E-2-E Nodal Delay

End-to-end delay is elapsed time from sending data at the source to receiving it at the destination. It is measured in seconds or milliseconds and sums processing, queuing, transmission, and propagation delays along the path.

$$d_{e2e} = \sum_{k \in \text{ links}} d_{\text{processing}}(k) + d_{\text{queue}}(k) + d_{\text{TX}}(k) + d_{\text{propagation}}(k)$$

- Processing: Error control, header reading
- Queuing: buffering
- Transmission: from the 1st bit to the last one, $\frac{\text{pkt size}}{\text{link rate}}$
- Propagation: usually distance over speed of light.

[17] Common Media Application Format

In modern networks an increase of R does affect only d_{TX}, the other delays keep the same size. The only thing we can change is the distance, we use CDNs to bring the content nearer to the users. Jitter is variation in packet delay or arrival spacing, commonly measured in milliseconds. Playback buffers absorb this variation by trading extra startup or end-to-end delay for smoother playout.

12.4 Buffer Dynamics

12.4.1 Rebuffering event

A playback buffer stores already downloaded media as seconds of future playable content. Rebuffering is a playback stall caused when this buffered duration reaches zero; during the stall, playback waits while new segments are downloaded.

$$B(t) = L \cdot T_s \qquad \text{where } \begin{cases} L = \#\text{segments} \\ T_s = \text{duration of a segment} \end{cases}$$

If the network goes down, slowly buffer occupation decreases.
In a QoE maximization approach, for a user is better to have low quality video than an interruption.

12.4.2 Playout management

When I empty the buffer I wait for the playout to restart until I received M segments (Hysteresis):

$$B(t) = M \cdot T_s \qquad \text{typically better for users if } L \geq M$$

We also set:

$$\frac{dB}{dt} = f_{IN} - f_{OUT}$$

Flows:

- f_{IN} Input Flow Rate, derived from network condition
  - Average $f_{IN} = \frac{D(T)}{T} = \frac{\bar{S}}{R_c}$
  - Instantaneous flow rate: $\frac{S(t)}{R_c} = \lim_{t \to 0} \frac{D(t)}{t}$
- f_{OUT} Output Flow Rate: speed of playout, considering 1x speed.

For a typical $f_{OUT} = 1$:

$$\frac{dB}{dt} = \begin{cases} \frac{S}{R_c} - 1 & \text{if playout going} \\ \frac{S}{R_c} & \text{if rebuffering, no playout} \end{cases}$$

The ideal condition is to have $S > R_c$ such that buffer is growing.

EXAM: Learn the Finite State Machine diagram and values → saw-tooth behavior.

12.4.3 Client Scoring-Policy Design

A QoE scoring policy maps measurable playback events to one utility value. It rewards segment quality and penalizes startup time, rebuffering duration, and visible quality changes; coefficients express their relative importance. Assume $K(n)$ is the quality of segment n, then we want a function to assign each $K(n)$ a score: ideally MOS.
The QoE needs to take into account:

- Per-segment video quality
- Penalize High Buffering Times
- Penalize frequent quality switching

Let's take $Q(n) = K_n$ or $Q(n) = R(K_n)$, and define Δ_n = duration of rebuff. during playout at segment n.
We can define $J(n)$ as the score for playout of segment n, takes into account quality switches and rebuffering:

$$J(n) = \lambda_1 K_n - \lambda_2 |K_n - K_{n-1}| - \phi(\Delta_n) \qquad \Rightarrow J(n) \propto K_n$$

To have a general score we define the final J by taking into account the first buffering event:

$$J = \sum_{n=1}^{N} J(n) - \lambda_3 T_{ST}$$

12.5 ABR Strategies

An ABR strategy is the client-side decision rule that chooses the bitrate of the next media segment. It must keep requested coding rate below predicted delivery capacity while preserving enough buffered playback time.

12.5.1 Throughput-based ABR

Throughput-based ABR selects the highest representation safely below an estimate of future network throughput. The safety factor $\eta < 1$ leaves margin for estimation error and short-term capacity drops. Goal: given an estimate of the current $S(t)$ let's check if segment arrives in time for playout.
Estimated Throughput:

$$S(n) = \frac{D(n)}{T_D(n)}$$

By using a smoothing filter (EMA):

$$\hat{S}(n) = \alpha S(n) + (1-\alpha)\hat{S}(n-1)$$

Decision Rule:

$$R(n) \leq \eta \hat{S}(n) \qquad \Longrightarrow \text{typical value } \eta = 0.85$$

Problem: here we totally ignore buffer status (buffer blindness).

12.5.2 Buffer-based ABR

Buffer-based ABR selects quality from current buffered playback duration rather than explicitly estimating throughput. Low occupancy triggers conservative rates; high occupancy permits larger rates. Strategy: select low quality if buffer is always empty, full quality if buffer is always full, linear behavior in the middle (2 thresholds to set). Problem: here we totally ignore network status.

12.5.3 Hybrid ABR strategy

Hybrid ABR combines throughput prediction, buffer occupancy, and a QoE objective. It evaluates candidate representations using expected quality, switching cost, and stall risk before requesting the next segment. Takes into account both strategies: it wants to maximize $J(n)$ as defined before. Pseudoalgorithm:

1. Set $J_{\min} = \infty$
2. $\forall l \in \{1 \cdots k\}$:
   Assuming constant $\hat{S}$ in the interval and $R = R(l)$, $\beta = \frac{\hat{S}}{R_c} - 1$, $B(t) = B_0 + \beta t$
   - if $\beta \geq 0$ set $\Delta = 0$ (won't empty while downloading)
   - if $\beta < 0$, compute $t_0 = \frac{B_0}{1 - \hat{S}R} = \frac{RB_0}{R - \hat{S}}$ emptying time, and check:
     - if $t_0 \geq T_s$ set $\Delta = 0$
     - if $t_0 < T_s$ set $\Delta = \frac{R}{\hat{S}}(T_s + MT_s)$

Server is totally unaware of what the client does, just serves what requested.

12.5.4 Learning-Based ABR

Learning-based ABR learns a representation-selection policy from observed states and rewards. Reinforcement learning models buffer and network measurements as state, representation choice as action, and QoE as reward.

12.5.5 Multiplayer Competition

Goal: guarantee the best possible average quality among all users.

Pathologies:

- BW unfairness
- Inefficiency (under-utilization)
- Instability (synchro-flicker effect)

What ABR Provides:

- Fairness
- Efficiency
- Stability

by using 'Implicit Coordination' mechanisms

12.5.6 Neural Video Coding

Neural video coding replaces one or more handcrafted codec blocks with learned transforms, motion models, and entropy models optimized jointly. Its rate term estimates coded bits, while its distortion term measures reconstruction or perceptual loss. Uses Learned Transforms (instead of DCTs), Learned Motion Estimation (instead of Block Matching), Learned Entropy Modeling to reach a Joint R/D optimization. There is a unified loss function to be able to use many differential metrics, like MS-SSIM and VMAF (useful as an indicator of Human perception):

$$\mathcal{L} = D(x, \hat{x}) + \lambda R(\hat{y})$$